

TASK FOR DeepSeek-R1 — Pure Math Functor `M`: Derive the Kinematic Autopilot

You are **`M`**, a deterministic mathematical-reasoning functor inside a multi-agent pipeline. You are NOT writing application code, UI, or I/O. You are deriving the **pure control-theory mathematics** that will be hand-compiled into an existing TypeScript inference engine for an on-device strength-and-grappling coaching app (**Athlete Kinetics**).

Your deliverable: model the athlete as a discrete-time **state vector** s_t , derive a **Flaw-Detection operator** F over a 3-week trailing window, and derive a deterministic **Control Action** u_{t+1} that auto-corrects the next 4-week training block. Everything you emit must reduce to closed-form arithmetic over fixed-length numeric arrays.

Read every constraint below before deriving. The interfaces and formulas in §3 are the **real, shipped boundaries** — your math must consume and produce these exact shapes, not idealized substitutes.

0. OUTPUT CONTRACT (obey exactly)

- Put **all** reasoning, scratch algebra, case analysis, and trade-off discussion inside `<thinking> ... </thinking>` tags. Be exhaustive in there.
- After** the closing `</thinking>` tag, emit only the final artifact, in this order:
 - `F` — Flaw Detection**: the closed-form formulas, then the TypeScript **interface/function signatures** (signatures + doc comments only — no bodies unless a body is a one-line pure expression).
 - `u\_{t+1}` — Control Action**: the closed-form mapping $F \rightarrow u$, then the TypeScript interface/function signatures.
 - Constant tables**: every coefficient, threshold, decay rate, and clamp you introduce, as named `const`` data tables (mirroring the style of `PHASE_MODS`` / `EXPERIENCE_TRIAGE`` in §3).
 - Invariant list**: the machine-checkable properties your math guarantees (determinism, monotone-conservatism, domain-closure, bounds).
- Formulas must be explicit. No "fit a regression" / "use an optimizer" / "apply a Kalman filter" hand-waves: if you want a slope, write the normal-equation closed form over the array; if you want a filter, write the scalar recurrence. Anything you name, you define.

1. SYSTEM CONSTRAINTS (non-negotiable — your math fails review if it violates these)

- 450 MB hard edge-compute envelope.** The entire app (RN/Hermes runtime + SQLite + intelligence layer) is audited against a 450 MB peak-RSS gate in CI (`tools/memory-audit``, currently ~450.1 MiB peak). Your algorithm runs in the **~100 MB transient** intelligence budget. No

allocation that scales with history beyond the bounded trailing window. Fixed-length `Float32Array`/`number[]` only.

- **Pure TypeScript runtime, fully on-device, offline.** Output is hand-translated to pure TS (strict mode). No native math libs, no BLAS/linear-algebra packages, no matrix types, no autodiff. Permitted primitives: `+`, `\*`, `/`, `Math.{min,max,abs,round,sqrt,log,exp,pow}`, comparisons, and array reduces. If you need `ln`/`sqrt`/`exp`, they are available; nothing else transcendental.

- **Determinism is a verified invariant.** No clock reads, no `Date.now()`, no randomness, no floating-point order dependence in reductions. Same persisted inputs $\Rightarrow$ byte-identical output (double-run deep-equality is a CI gate). Use **fixed rounding** exactly like the codebase: `round2(n)=Math.round(n\*100)/100`, `round4(n)=Math.round(n\*1e4)/1e4`. RPE rounds to the nearest 0.5 (`Math.round(x\*2)/2`); kg rounds to 2.5 kg plates.

- **SQLite (WAL mode) read/write limits.** Single-writer WAL; readers see a snapshot. The flaw detector is a **read-only trailing-window scan** — it consumes at most the last **21 daily rows** of `state\_vector` (a clustered `WITHOUT ROWID` range scan, like the existing 14-day trend read) plus the windowed `set\_record`/`niggle` aggregates. It must NOT aggregate raw sets at read time beyond what the rollups below already expose. The control action `u\_{t+1}` must be applied as a **single bounded upsert** (a side-car row keyed on a parent PK, FK-cascaded — the established `slot\_override` / `set\_prefix` pattern), never a multi-statement migration or a hot-path table rewrite. Absent calendar days are **zero-load days**, not missing rows (gap-tolerant calendar `RANGE` windows — see §3).

- **No LLM at runtime (see §5).**

2. THE STATE VECTOR `s\_t`

Model the athlete on calendar day `t` (ISO `YYYY-MM-DD`, lexicographic = chronologic) as:

...

$s_t = (V_t, I_t, C_t, \Delta E_t)$

...

Each component already has a **persisted, deterministic source** in the schema. Use these sources — do not invent new sensors.

- **`V\_t` — Mechanical Volume (Raw & Effective Tonnage).**

- **Raw tonnage*** (per set): `tonnage\_kg` = `reps` · `load\_kg` (a SQLite `GENERATED ... STORED` column), summed per day into `mech\_daily.tonnage\_kg`.

- **Effective tonnage*** (per set): `effective\_load\_kg` from the condition fold (§3 `calculateEffectiveLoad`): `effectiveLoad` = `baseLoad` · $\prod$ `difficultyModifier`. Persisted per set in `set\_prefix`. **Bifurcation invariant:** raw tonnage feeds ACWR/readiness; effective tonnage is a **separate** recoverable signal — never conflate them.

- **Rolling load*:** `acute\_load\_kg` = mean daily tonnage over the trailing 7 calendar days; `chronic\_load\_kg` = mean over 28; `acwr` = `acute/chronic` (defined only when `chronic\_load\_kg` > 1.0).

- **`I\_t` — Injury / Niggle severity matrix.** Source: `niggle(region,

severity, reported_at_ms)', append-only. `region ∈ JOINTS` (9 joints, §3).
 `severity ∈ [1,10]` integer. Map to structures via `PATTERN\_JOINTS`
 (movement-pattern → loaded joints). The operative/halt thresholds are
 experience-weighted via `EXPERIENCE\_SEVERITY[trainingAge]`
 (§3). Treat `I\_t` as a per-joint severity signal over the window (you
 choose the decay/aggregation, but justify it and keep it bounded).
 - **`C\_t` — Daily Condition / Readiness.** Source:
 `state\_vector.readiness\_score ∈ [0,100]` and its four sub-components,
 computed by the materializer (§3): `readiness = 0.35·hrv + 0.30·load +
 0.25·sleep + 0.10·spo2`. Each sub-component ∈ [0,100], 50 = neutral/no-
 data.
 - **`ΔE\_t` — RPE Error Delta.** The signed gap between **prescribed**
 effort and **reported actual** effort, per movement pattern:
 ...

$$\Delta E(\text{slot}) = \text{rpe_actual} - \text{rpe_target}$$

$$= \text{set_record.rpe} - \text{PlannedSlotPlan.target_rpe}$$
 ...
 `rpe\_target` is the planned slot effort (§3 `PlannedSlotPlan.target_rpe ∈
 [5.0,10.0]`). `rpe_actual` is the logged `set_record.rpe ∈ [0,10]`. **Sign
 convention:** `ΔE > 0` ⇒ the athlete reported the work *harder than
 programmed* (under-recovery / capacity deficit for that pattern); `ΔE <
 0` ⇒ easier than programmed (latent headroom). Aggregate per
 `MovementPattern` (and, via `PATTERN\_TO\_CATEGORY` /
 `PATTERN\_JOINTS`, per category/joint).

3. CODEBASE STATE — exact interfaces & mathematical boundaries

These are extracted verbatim (comments trimmed) from
 `packages/inference` and `packages/core-db`. Your derivations must
 type-check against them.

3.1 Movement taxonomy (the index set of your per-pattern math)

```
``ts
export const MOVEMENT_PATTERNS = [
  'squat','hinge','push_h','push_v','pull_h','pull_v',
  'lunge','carry','rotation','isolation','locomotion',
] as const;
export type MovementPattern = (typeof MOVEMENT_PATTERNS)
[number];

export const PATTERN_TO_CATEGORY: Record<MovementPattern,
TaxonomyCategory> = {
  squat:'squat', hinge:'hinge', push_h:'push', push_v:'push',
  pull_h:'row', pull_v:'row', lunge:'unilateral', carry:'accessory',
  rotation:'core', isolation:'accessory', locomotion:'cardio',
};
...

```

3.2 `PlannedSlotPlan` — one prescribed working slot (the thing `u\_{t+1}` rewrites)

```
``ts
export interface PlannedSlotPlan {

```

```

slot_index: number; // 1-based
movement_id: number;
sets: number; // 1..10 (schema CHECK)
reps: number; // 1..30
target_rpe: number; // 5.0..10.0 (PRESCRIBED effort — the ΔE
reference)
  applied_prefixes?: readonly MovementPrefix[]; // Phase-13 condition
tokens
}
...

```

3.3 RPE scale & boundaries (single source of truth)

- Logged/raw RPE domain (`set\_record.rpe`, `session.session\_rpe`, `target\_avg\_rpe`): ****REAL BETWEEN 0 AND 10**** (nullable).
- Prescribed slot RPE (`PlannedSlotPlan.target\_rpe`): ****[5.0, 10.0]****, the block generator floors at 5.0 and rounds to 0.5.
- Athlete effort ceiling `base\_rpe\_cap`: ****[5.0, 10.0]**** in 0.5 steps**.
- Hard caps already enforced (monotone-conservative chain): ****rehab ≤ 7.0**, **beginner ≤ 8.5**, **any flagged/restrictive report ≤ 8.0**** (**FLAGGED_RPE_CEILING**).
- Reps-in-reserve identity used for %1RM: ****RIR = 10 – rpe****.
- Discrete output RPE-cap vocabulary (**RPE_CAP_LITERALS**): **['6.5','7.0','7.5','8.0','8.5','9.0','9.5','10.0']**.

3.4 `effectiveLoad` — the condition fold (pure, order-independent)

```

...ts
export interface MovementPrefixCondition {
  prefixName: MovementPrefix;
  cnsLoadModifier: number; // > 0, 1.0 = neutral
  stabilityRequirementModifier: number; // > 0, 1.0 = neutral
  difficultyModifier: number; // > 0, 1.0 = neutral
}

export interface EffectiveLoad {
  baseLoad: number; // clamp(base, ≥0)
  effectiveLoad: number; // baseLoad · ∏ difficultyModifier ("feels-like"
kg)
  cnsLoad: number; // baseLoad · ∏ cnsLoadModifier
  stabilityDemand: number; // ∏ stabilityRequirementModifier
(dimensionless)
  appliedPrefixes: readonly MovementPrefix[];
}

// safeBase = (isFinite(baseLoad) && baseLoad>0) ? baseLoad : 0
// each modifier m>0 else coerced to 1 (guards the product)
// return { baseLoad: round2(safeBase),
//   effectiveLoad: round2(safeBase·∏diff),
//   cnsLoad: round2(safeBase·∏cns),
//   stabilityDemand: round4(∏stab), appliedPrefixes }
export function calculateEffectiveLoad(
  baseLoad: number,
  prefixModifiers: readonly MovementPrefixCondition[],
): EffectiveLoad;
...

```

3.5 `AdjustmentVector` — the existing daily control output (your `u` must compose with this)

```
``ts
export const LOAD_MODIFIER_LITERALS =
['0.80','0.85','0.90','0.95','1.00','1.05'] as const;
export const SET_MODIFIER_LITERALS = ['-2','-1','0','1'] as const;
export const RPE_CAP_LITERALS =
['6.5','7.0','7.5','8.0','8.5','9.0','9.5','10.0'] as const;

export interface AdjustmentVector {
  load_modifier: number; // multiplier on today's planned working
weights
  set_modifier: number; // delta on planned working sets per movement
  rpe_cap: number; // hard RPE ceiling for the session
  coaching_cue: string; // one blunt mechanical-rationale sentence
(12..140 chars)
}

export const NEUTRAL_ADJUSTMENT: AdjustmentVector =
{ load_modifier:1.0, set_modifier:0, rpe_cap:9.0, coaching_cue:'...
execute as written.' };

export const SET_MODIFIER_FLOOR = -3; // penalties never stack below
this
// Final defensive clamp (idempotent, no-op on valid vectors):
export function clampAdjustment(v: AdjustmentVector):
AdjustmentVector;
// load_modifier: max(0, v.load_modifier)
// set_modifier : max(SET_MODIFIER_FLOOR, v.set_modifier)
// rpe_cap : min(10, max(0, v.rpe_cap))
...

```

3.6 `Guardrail` — the restrictive-report shape (for safety composition)

```
``ts
export interface Guardrail {
  load_multiplier: number; // ≤ 1 when restrictive
  set_delta: number; // ≤ 0 when restrictive
  rpe_cap_max: number; // < 10 when restrictive
  halt: boolean; // hard stop — NEVER relaxed by re-derivation
  follow_up: string | null;
}
...

```

3.7 Experience weighting (DOMS-vs-structural — already shipped)

```
``ts
export const EXPERIENCE_SEVERITY: Record<TrainingAge,
{triageMin:number;haltMin:number}> = {
  beginner:{triageMin:5,haltMin:8}, intermediate:{triageMin:4,haltMin:8},
  advanced:{triageMin:3,haltMin:7}, elite:{triageMin:3,haltMin:6},
};
export const EXPERIENCE_TRIAGE: Record<TrainingAge,

```

```

{loadScale:number;capDelta:number;extraSetCut:number}> = {
  beginner:{loadScale:1.2,capDelta:2.0,extraSetCut:0},
  intermediate:{loadScale:1.0,capDelta:0.0,extraSetCut:0},
  advanced:{loadScale:0.9,capDelta:-0.5,extraSetCut:0},
  elite:{loadScale:0.8,capDelta:-1.0,extraSetCut:1},
};
// TrainingAge = 'beginner'|'intermediate'|'advanced'|'elite'
...

### 3.8 Injury structures (`l_t` boundary)

``ts
export const JOINTS = ['knee','hip','ankle','shoulder','elbow','wrist',
  'lower_back','spine','neck'] as const;      // == niggle.region CHECK
domain
export const PATTERN_JOINTS: Record<MovementPattern, readonly
Joint[]> = {
  squat:['knee','hip'], hinge:['hip','lower_back'],
  push_h:['shoulder','elbow'], push_v:['shoulder','elbow'],
  pull_h:['shoulder','elbow'], pull_v:['shoulder','elbow'],
  lunge:['knee','hip','ankle'], carry:['shoulder'], rotation:['spine'],
  isolation:[], locomotion:['knee','ankle'],
};
// niggle row: { regioneJOINTS, severity INTEGER 1..10, reported_at_ms
INTEGER }
...

### 3.9 Block-template math (what `u_{t+1}` perturbs)

``ts
// Per-objective base scheme; rpeWave is weeks 1..3, week 4 = deload.
const SCHEMES: Record<Objective,
{reps:number;sets:number;rpeWave:[number,number,number]}> = {
  strength:{reps:5,sets:4,rpeWave:[7.5,8.0,8.5]}, power:
{reps:3,sets:5,rpeWave:[7.0,7.5,8.0]},
  hypertrophy:{reps:10,sets:4,rpeWave:[7.5,8.0,8.5]}, endurance:
{reps:15,sets:3,rpeWave:[6.5,7.0,7.5]},
  gpp:{reps:8,sets:3,rpeWave:[7.0,7.5,8.0]}, hybrid:
{reps:5,sets:4,rpeWave:[7.5,8.0,8.5]},
  rehab:{reps:12,sets:3,rpeWave:[6.0,6.5,7.0]}, weight_loss:
{reps:12,sets:3,rpeWave:[7.0,7.5,8.0]},
};
const PHASE_MODS: Record<MacroPhase,
{reps:number;rpe:number;sets:number}> = {
  gpp:{reps:2,rpe:-0.5,sets:0}, hypertrophy:{reps:3,rpe:0,sets:0},
  volume:{reps:0,rpe:0,sets:1}, peak:{reps:-2,rpe:0.5,sets:0},
};
export const SCHEMA_FATIGUE_COST:
Record<SchemaType,Record<MacroPhase,number>> = {
  LINEAR:{gpp:1.0,hypertrophy:1.1,volume:1.2,peak:1.2}, WAVE:
{gpp:1.1,hypertrophy:1.2,volume:1.3,peak:1.3},
  STEP:{gpp:1.1,hypertrophy:1.2,volume:1.4,peak:1.3}, APRE:
{gpp:1.3,hypertrophy:1.4,volume:1.5,peak:1.6},
};
export const HYBRID_TAX_THRESHOLD = 1.3; // ≥ strips 1 accessory
set, ≥1.5 strips 2

```

```

export const OVERREACH_ACWR = 1.5;      // peak-block deadlift auto-
regulation gate
export const BLOCK_WEEKS = 4;          //
accumulation→intensification→realization→deload

// RPE/rep → %1RM (Epley) and physical target weight (2.5 kg plate
rounding):
export const targetPct = (reps:number, rpe:number): number =>
  1 / (1 + (reps + Math.max(0, 10 - rpe)) / 30);
export const targetLoadKg = (oneRmKg:number, reps:number,
rpe:number): number =>
  Math.max(0, Math.round((oneRmKg * targetPct(reps,rpe)) / 2.5) * 2.5);
...

### 3.10 Readiness materializer (`C_t` boundary) & windowed features
(`V_t` boundary)

...

hrv_z      = (ln_rmssd - baseline_mean) / baseline_sd    -- baseline
over [28d..1d) PRECEDING
hrv_component = clamp(50 + 25·hrv_z, 0, 100)              -- 50 if null
load_component = acwr<0.8 : clamp(100 - (0.8-acwr)·125, 0, 100) --
detraining penalty 125/unit
               0.8≤acwr≤1.3 : 100                        -- sweet spot
               acwr>1.3 : clamp(100 - (acwr-1.3)·200, 0, 100) -- spike
penalty 200/unit
               acwr null : 50
sleep_component= clamp((sleep_eff_pct - 65)·(100/30), 0, 100) -- 50
if null
spo2_component = clamp((spo2_night_mean - 90)·(100/7), 0, 100) --
50 if null
readiness      = round(0.35·hrv + 0.30·load + 0.25·sleep + 0.10·spo2, 1)

acute_load_kg  = Σ tonnage_kg over RANGE [6 PRECEDING, CURRENT]
/ 7.0 -- absent day = 0
chronic_load_kg = Σ tonnage_kg over RANGE [27 PRECEDING,
CURRENT] / 28.0
acwr           = chronic_load_kg > 1.0 ? acute_load_kg / chronic_load_kg :
NULL
...

`StateVectorRow` (the only inference read surface, one clustered PK row
per day):

```ts
export interface StateVectorRow {
 date: string; readiness_score: number;
 hrv_component: number; load_component: number; sleep_component:
number; spo2_component: number;
 acwr: number|null; acute_load_kg: number|null; chronic_load_kg:
number|null;
 ln_rmssd: number|null; hrv_z: number|null;
 sleep_efficiency_pct: number|null; spo2_night_mean: number|null;
 computed_at_ms: number;
}
...

```

---

## ## 4. YOUR DERIVATION TASK

Derive two operators. Treat the next block as a 4-week horizon; the trailing window is  **$W = 21$  days** (3 micro-weeks), gap-tolerant on the calendar exactly like the 7/28-day windows above.

### ### 4.1 `F` — Flaw-Detection operator (the observer)

Define `F` as a pure map from the windowed history to a **per-pattern flaw signal**:

...

$F : \{ \text{StateVectorRow}[\leq 21], \text{perPattern}\Delta E[\leq 21], \text{niggleRows}[\text{window}] \} \rightarrow \text{FlawReport}$

...

Requirements:

- Produce, per `MovementPattern p` (11 of them), a bounded **deficit score**  $\varphi_p \in [-1, 1]$  (or a documented bounded range) built from:
  - the **persistence and magnitude** of  $\Delta E_p > 0$  across the window (your example case: persistent  $\Delta E > 0$  on `hinge`  $\Rightarrow$  posterior-chain deficit). Specify the exact aggregation — e.g. a fixed- $\lambda$  EWMA or a windowed mean with a `persistence` count — as a closed-form recurrence/sum over the array. No library calls.
  - a **condition-weighted** adjustment using `cnsLoad` / `stabilityDemand` from §3.4 where prefixes were applied (a hard  $\Delta E > 0$  under a low-CNS condition is more diagnostic than under an `Earthquake Bar`).
  - the **joint-injury coupling** via `PATTERN\_JOINTS`  $\cap$  `I\_t`: an injured loaded joint must **attenuate** a "train it harder" deficit reading (you never chase a deficit through a niggle).
  - a **trend** term (sign of change across the window) — give the explicit 2-point or least-squares-slope closed form over the daily array.
- Emit a discrete **flaw class** per pattern from a fixed enum you define (e.g. `capacity\_deficit` | `latent\_headroom` | `fatigue\_masking` | `neutral`), via deterministic thresholds on  $\varphi_p$  (thresholds as a named constant table).
- Confidence gating: a pattern with too few logged sets in the window (below a fixed `MIN\_OBSERVATIONS`) returns `neutral` (no action on thin data — mirror the existing 0.55 confidence-gate philosophy).
- Output a `FlawReport` interface: per-pattern  $\{ \text{pattern}, \phi, \text{flawClass}, \text{observations} \}$  plus a window-level summary.

### ### 4.2 $u_{t+1}$ — Control Action (the deterministic controller)

Define  $u_{t+1} = g(\text{FlawReport}, \text{profile}, \text{macroPhase})$  that auto-corrects the **next** block. It must be expressible as a perturbation of the §3.9 generator tables and must **degrade** to the existing `AdjustmentVector` domain at the daily grain. Concretely, derive:

- A **per-pattern correction**  $u_p = \{ dLoad_p, dSet_p, dRpe_p, \text{prefBias}_p \}$  where:
  - $dRpe_p$  adjusts the prescribed `rpeWave` / slot `target\_rpe` for



pattern `p` (a `capacity\_deficit` lowers prescribed effort to close `ΔE`; `latent\_headroom` may raise it — but **never above** the §3.3 ceilings or `base\_rpe\_cap`).

- `dSet\_p` adds/removes **corrective accessory volume** for the deficit's category (`PATTERN\_TO\_CATEGORY`), **budget-bounded** by `SCHEMA\_FATIGUE\_COST` / `HYBRID\_TAX\_THRESHOLD` (no CNS overspend) and floored at `SET\_MODIFIER\_FLOOR`.

- `dLoad\_p` is snapped onto `LOAD\_MODIFIER\_LITERALS` when surfaced as a daily `AdjustmentVector.load\_modifier`.

- `prefBias\_p ∈ {-1,0,+1}` biases the deterministic substitution router toward/away from the pattern (reuse the existing preference sentiment scale).

- A **controller law** mapping  $\varphi_p \rightarrow u_p$ : give the explicit transfer function (proportional, or P-with-deadband — define the deadband so small  $|\varphi_p| \Rightarrow$  zero action and the system doesn't hunt). State the **gain constants** as a named table. Prove the loop is **contractive** / **non-oscillating** within the safe envelope (bounded gain  $\times$  the existing monotone clamps).

- **Safety / composition invariants** (must hold by construction):

1. **Monotone-conservative under flags:** if any `Guardrail` is restrictive or `l\_t` flags a loaded joint,  $u_{t+1}$  may only reduce load/sets/effort for affected patterns — never raise them.

2. **Domain closure:** the daily projection of  $u_{t+1}$  lands inside the `AdjustmentVector` literal domains after `clampAdjustment`, and block-level `target\_rpe` stays in `[5.0,10.0]` rounded to 0.5.

3. **Halt supremacy:** a recorded `halt` is never relaxed by  $u_{t+1}$ .

4. **Bounded authority per block:** total corrective volume added across the block  $\leq$  a fixed cap (state it);  $u$  cannot move the athlete more than one "step" of load/effort per block (anti-windup).

5. **Volume bifurcation respected:** corrections reason over **effective** tonnage where relevant but never rewrite the **raw** `mech\_daily`/ACWR pipeline.

- Output a `ControlAction` interface (`Record<MovementPattern, PatternCorrection>` + block-level scalars) and the pure signature `deriveControlAction(report: FlawReport, profile: UserProfile, macroPhase: MacroPhase): ControlAction`.`

---

## ## 5. NO LLM AT RUNTIME — HARD COMPILE TARGET

The shipped app runs **no generative model**. A sub-3B on-device SLM was trialed and **removed** (it held the output grammar but could not execute the numeric rule table — constant outputs regardless of input). The only ML artifact on device is a ~23 MB sentence-embedding model used solely for routing free-text reports by cosine similarity; it produces **no numbers your math may depend on**. Therefore:

- Your `F` and  $u_{t+1}$  must be **pure deterministic arithmetic** that hand-translates to TypeScript operating over plain numeric arrays (`number[]` / `Float32Array`) — **no inference call**, no learned weights, no training step, no runtime model of any kind.

- Every coefficient is a **human-reviewable constant** you emit in §3-style tables (like `PHASE\_MODS`), not a fitted parameter.

- No operation may require more than the §1 primitive set, must be  $O(\text{window})$  time and  $O(1)$ -beyond-window space, and must satisfy the

determinism/rounding rules in §1.

Begin. Reason inside ``<thinking>``; then emit ``F``, ``u_{t+1}``, the constant tables, and the invariant list.

#### Thought for 169 seconds

We need to design a deterministic Flaw-Detection operator  $F$  and a Control Action  $u_{t+1}$  for an on-device training autopilot. Constraints: pure arithmetic over bounded arrays, determinism, fixed rounding. We'll produce closed-form formulas, then TypeScript signatures, constant tables, and invariants.

We must model the athlete state vector  $s_t$  as given:  $V_t$  (volume, ACWR),  $I_t$  (injury niggles),  $C_t$  (readiness),  $\Delta E_t$  (RPE error per pattern). The flaw detection uses trailing 21-day window (calendar days, gap-tolerant: absent days are zero-load days, but for  $\Delta E$  we need logged sets; we handle missing data with "neutral" if insufficient observations). We'll produce per-pattern  $\phi$  score and flaw class. The control action perturbs next 4-week block (weeks 1-3 accumulation, week 4 deload) based on flaw report, profile (training age, etc.), and current macro phase. The  $u_{t+1}$  must be expressible as per-pattern corrections that feed into daily AdjustmentVector (load modifier, set modifier, rpe cap) and block-level target\_rpe adjustments.

We must respect all existing interfaces, literals, and guardrails.

Let's think through the math.

First, define  $\Delta E_p(t)$ : for a given movement pattern  $p$ , on day  $t$ , we have aggregate RPE error. But there could be multiple sets/slots for a pattern. We need per-pattern daily  $\Delta E$  aggregation: average  $\Delta E$  over all slots of that pattern on that day, or maybe weighted by set count? The system has PlannedSlotPlan and set\_record. For simplicity, we can compute per-pattern daily  $\Delta E$  as mean of slot  $\Delta E$ , but we must handle no sessions for that pattern on a day: missing day = no observation (null). We'll treat missing as no update to trend, but we need a windowed aggregation over 21 days. So we'll create for each pattern  $p$  a daily series  $e_p(d)$  where  $d$  is calendar day in window;  $e_p(d)$  = average  $\Delta E$  over all sets of pattern  $p$  performed on day  $d$ , if any; else null.

We need a deficit score  $\phi_p \in [-1,1]$ . Positive  $\phi \Rightarrow$  capacity deficit (athlete overexerted relative to prescription, under-recovery). Negative  $\phi \Rightarrow$  latent headroom (easier than prescribed). We'll design  $\phi$  such that persistent positive  $\Delta E$  (harder than programmed) yields positive  $\phi$ , negative vice versa. The magnitude captures severity and persistence.

We'll incorporate:

- Persistence: count of days with  $\Delta E > 0$  vs  $\Delta E < 0$ , or a decay-weighted sum.
- Magnitude: absolute  $\Delta E$  values.

- Condition weighting: when CNS load or stability demand is high, a given  $\Delta E > 0$  is less indicative of a capacity deficit? Actually the requirement: "a hard  $\Delta E > 0$  under a low-CNS condition is more diagnostic than under an Earthquake Bar." So we weight the  $\Delta E$  by  $(1 / (\text{cnsLoadModifier factor?}))$  or maybe by how much the condition likely inflated the RPE. The idea: if the movement used prefixes that increase difficulty (e.g., stability demand), then a high RPE might be due to the challenge, not a true deficit. So we should attenuate the signal when difficulty modifiers are high. We can define a weight  $w_p(t) = 1 / (\text{effectiveLoad multiplier?})$ . Actually effectiveLoad multiplier is product of difficultyModifiers; high difficulty modifier means the load feels heavier, so a given RPE might be less indicative of under-recovery. Conversely, low difficulty modifier (e.g., stable movement) means a high RPE is more alarming. So we could weight  $\Delta E$  by  $1 / (\text{difficulty product})$ , but careful:  $\Delta E$  is RPE error, not load. RPE is effort rating; if the movement is inherently more demanding (e.g., Earthquake Bar), the athlete might report higher RPE for the same relative effort, which could be "noise". So we can discount the signal when stability/difficulty modifiers are high. We'll define an attenuation factor:  $\text{att}_p(t) = 1 / (\max(1, \text{stabilityDemand} / \text{baseline?}))$  but simpler: use a fixed reference. Since we have  $\text{cnsLoad}$  and  $\text{stabilityDemand}$  dimensionless numbers; we can compute condition complexity  $c_p(t) = (\text{cnsLoad} / \text{baseLoad}) * \text{stabilityDemand}$ ? But we have from EffectiveLoad the effectiveLoad ratio =  $\text{effectiveLoad} / \text{baseLoad}$  = product of difficultyModifiers. So high difficulty modifier  $\Rightarrow$  high effectiveLoad/baseLoad. So we can define weight =  $1 / (\text{effectiveLoad} / \text{baseLoad})$  clamped maybe. So  $w_p(t) = 1 / \max(1, \text{effectiveLoadRatio})$ . If effectiveLoadRatio is 1 (no prefixes), weight=1; if it's  $>1$ , weight $<1$ . So we attenuate  $\Delta E$  magnitude.

But we need to get effectiveLoad for each set. The state vector  $s_t$  doesn't store per-set effectiveLoad directly, but we can compute from  $\text{set\_record}$  and prefixes. Since we are reading from existing data, we must have access to  $\text{set\_prefix}$  and prefix modifiers. The problem says "the windowed  $\text{set\_record}$ /niggle aggregates" and "bounded trailing window". It says the flaw detector must NOT aggregate raw sets at read time beyond what the rollups below already expose. Wait: constraint: "It must NOT aggregate raw sets at read time beyond what the rollups below already expose." The "rollups below" refer to the given interfaces: we have per-day state vector with readiness, ACWR, etc., and presumably we can access the daily set records with effective load? But they said the flaw detector consumes at most the last 21 daily rows of  $\text{state\_vector}$  plus the windowed  $\text{set\_record}$ /niggle aggregates. The state vector doesn't include per-pattern  $\Delta E$  or per-set effective load. So we must assume the detector has access to the  $\text{set\_record}$  rows within the window, but the constraint says it must not aggregate raw sets beyond what the rollups expose. That seems contradictory. Actually: "It must NOT aggregate raw sets at read time beyond what the rollups below already expose." "the rollups below" refers to the earlier bullet points? Or maybe they refer to the "windowed  $\text{set\_record}$ /niggle aggregates". They say: "The flaw detector is a read-only trailing-window scan — it consumes at most the last 21 daily rows of  $\text{state\_vector}$  (a clustered WITHOUT ROWID range scan, like the existing 14-day trend read) plus the windowed  $\text{set\_record}$ /niggle aggregates." So the detector can read 21 daily  $\text{state\_vector}$  rows, and also windowed aggregates of  $\text{set\_record}$  and niggle. Those aggregates must be precomputed or simple. The constraint about "must NOT aggregate raw sets at read time beyond what the rollups below already expose" likely means we cannot scan all raw  $\text{set\_record}$  rows to compute something like per-set effective load; we must use pre-aggregated per-day per-pattern sums. But the existing schema doesn't have per-day per-pattern  $\Delta E$  aggregates, but we can assume we can compute a simple aggregate like average  $\Delta E$  per pattern per day over the window by scanning the  $\text{set\_record}$  rows (which are limited to 21 days). The memory budget is ~100 MB, scanning a few hundred rows is fine. But they said "must NOT aggregate raw sets at read time beyond what the rollups below already expose". This might be a performance constraint: avoid expensive per-set aggregation at query time. However, the system likely already stores daily rollups for tonnage, etc. But  $\Delta E$  is per slot, not pre-aggregated. Since we're designing the math, we can define that the flaw detector computes, for each pattern  $p$ , over the 21-day window, a daily summary of  $\Delta E$ : we scan the  $\text{set\_record}$  rows for those days, group by pattern and day, compute mean  $\Delta E$ . That's a simple aggregation; it's within the allowed window. We'll assume it's okay, as long as we define it as a deterministic scan. The constraint might be to avoid heavy computation like least-squares

over raw sets, but we can still do it. I'll note that we'll compute per-day per-pattern average  $\Delta E$ , count of sets, and also average effectiveLoad modifier (ratio) for weighting. We'll derive closed-form sums.

So, for each pattern  $p$ , for each day  $d$  in window (0..20 days before  $t$ ), we compute:

- $n_p(d)$  = number of sets logged for pattern  $p$  on day  $d$  (0 if none).
- $e_p(d)$  = mean  $\Delta E$  over those sets, if  $n_p(d) > 0$ ; else null.
- $w_p(d)$  = average of  $1 / \max(1, \text{effectiveLoadRatio})$  over those sets, where  $\text{effectiveLoadRatio} = \text{effectiveLoad} / \text{baseLoad}$  (if  $\text{baseLoad} > 0$ , else 1). This is a dimensionless attenuation. If no sets,  $w_p(d)=0$  (or ignore).

We also need the niggle coupling:  $l_t$  is per-joint severity. For each pattern, we define a joint severity sum or max:  $j_p(d) = \max$  severity over joints in  $\text{PATTERN\_JOINTS}[p]$  on day  $d$ , from niggle rows. If none, 0. We'll incorporate that to attenuate positive  $\phi$  (deficit) because we don't chase deficits through injury.

Now, design  $\phi_p$ . We want a bounded score between -1 and 1.

We can define a raw deficit signal  $D_p$  = some aggregation of  $(w_p(d) * e_p(d) * \text{sign})$  with persistence. Use exponential moving average (EMA) with a fixed decay factor  $\alpha$  ( $0 < \alpha < 1$ ) over the 21 days, but we need deterministic recurrence over array. We'll define a decay-weighted sum and a normalization. We'll treat days in chronological order (oldest to newest). We'll compute:

Let the window have  $D$  days (21, but could be less if at start). For each day  $i=0..20$  ( $i=0$  oldest,  $i=20$  most recent, day  $t$ ). Define:

- $e_i = e_p(\text{day}_i)$ ,  $w_i = w_p(\text{day}_i)$ ,  $n_i = n_p(\text{day}_i)$  (count),  $j_i = \text{joint\_max\_severity}(\text{day}_i)$ .

We'll compute a weighted deficit accumulation:

$S_{\text{pos}} = \sum_{\{i: e_i > 0\}} \lambda^{(20-i)} * w_i * \min(e_i, \text{some cap?}) * (1 - j_i/10)$  //  $j_i$  reduces signal  
 $S_{\text{neg}} = \sum_{\{i: e_i < 0\}} \lambda^{(20-i)} * w_i * |e_i| * (1 - j_i/10)$  // if  $e_i$  negative, that's headroom; we might not attenuate with injury? Actually injury would make us more cautious, but if athlete has headroom (easier than programmed) despite injury, that's not a deficit. So we could treat headroom as positive signal for "latent headroom" even with injury? Might be misleading. Better to not attenuate headroom by injury, because if athlete is injured but still reporting easier than programmed, that may indicate capacity but we should still be cautious. However, the requirement: "an injured loaded joint must attenuate a 'train it harder' deficit reading". So only for deficit ( $\Delta E > 0$ ) we attenuate; for headroom ( $\Delta E < 0$ ) we might not attenuate, or we could still limit because we don't want to increase load on injured joint. I'll only attenuate positive deficit accumulation.

So define weighted positive sum:

$P_p = \sum_{\{i: e_i > 0\}} \lambda^{(20-i)} * w_i * \min(e_i, E_{\text{max}}) * \max(0, 1 - j_i / J_{\text{max}})$  where  $J_{\text{max}} = 10$  (severity scale), and  $E_{\text{max}}$  is maybe 3 (max RPE error plausible, since RPE range is 0-10, but error could be up to 5 if target 5 and actual 10, but capped by experience ceilings, but we can cap at 2.5 to be safe). I'll set  $E_{\text{max}} = 3.0$ .

Similarly, negative sum for headroom ( $\Delta E < 0$ ):

$N_p = \sum_{\{i: e_i < 0\}} \lambda^{(20-i)} * w_i * \min(|e_i|, E_{\text{max}})$

Then we also compute total weight sum (for normalization):  $W_p = \sum_{\{i: n_i > 0\}} \lambda^{(20-i)}$  // count days with any observation, not weighted by  $w_i$ ? We'll normalize by the max possible sum of weights (geometric series). The maximum sum of  $\lambda^{(20-i)}$  over 21 days is  $S_{\text{max}} = (1 - \lambda^{21}) / (1 - \lambda)$ .

We'll normalize  $P_p$  and  $N_p$  by  $S_{\text{max}}$ , then  $\phi_p = (P_p - N_p) / S_{\text{max}}$ ? But that would range roughly  $-E_{\text{max}}$  to  $E_{\text{max}}$ , not -1 to 1. We'll instead compute a scaled ratio:  $\phi_p = (P_p - N_p) / (S_{\text{max}} * E_{\text{scale}})$  where  $E_{\text{scale}}$  is some factor to bound to  $[-1, 1]$ . Simpler: define  $\phi_p = \tanh(k * (P_p - N_p))$  but  $\tanh$  not primitive!  $\text{Math.tanh}$  not in allowed? They said  $\text{Math}$ .

{min,max,abs,round,sqrt,log,exp,pow}.  $\tanh = (\exp(2x)-1)/(\exp(2x)+1)$ , so we can use  $\exp$ . But can we use  $\exp$ ? They said "If you need  $\ln/\sqrt{\exp}$ , they are available; nothing else transcendental."

Yes,  $\exp$  is available. So  $\tanh$  is allowed via  $\exp$ . But we need determinism and rounding: we'll use

round4. So we can do  $\phi_p = \tanh(\text{gain} * (P_p - N_p) / \text{norm})$  where norm could be  $S_{\max}$ , to squash to  $[-1,1]$ . But tanh of large values saturates. That's fine. We'll define gain = 2.0 to make it sensitive.

But we also need a trend term. We'll compute slope over the window to capture trend: if deficit is increasing ( $\Delta E$  getting more positive over time), it's more concerning. We'll compute a simple least-squares slope of  $e_i$  (weighted) over days index (0..20). We can compute slope  $m = (N \sum (x_i y_i) - \sum x_i \sum y_i) / (N \sum x_i^2 - (\sum x_i)^2)$  with  $x_i = i$  (0..20),  $y_i = e_i$  (or weighted?). Use weighted if  $n_i > 0$ , else omit? For trend, we want recent days more important? We'll compute a weighted linear regression with weights  $\lambda^{(20-i)}$ . But least-squares slope closed form with weights is possible, but we must keep it simple. We can compute a simple 2-point slope: difference between average of last 7 days and average of preceding 7 days, normalized. That's a trend signal  $t_p \in [-1,1]$  (using tanh again). We'll compute recent mean  $e_{\text{recent}} = \text{average of } e_i$  for days  $i=14..20$  (last 7 days) if observations exist, else 0; older mean  $e_{\text{older}} = \text{average over days } 0..6$  (oldest 7 days). Then  $\text{trend\_raw} = e_{\text{recent}} - e_{\text{older}}$ . Then  $t_p = \tanh(\text{trend\_raw} / T_{\text{scale}})$  with  $T_{\text{scale}} = 1.0$  (since  $\Delta E$  is small). We'll ensure deterministic.

Finally, combine into final  $\phi_p = \text{combine}(\text{base } \phi_{\text{raw}} \text{ from accumulation, and trend } t_p, \text{ and maybe readiness context})$ . The requirement: "condition-weighted adjustment using cnsLoad/stabilityDemand". We already used  $w_p(d)$  which captures difficulty modifiers per set; that's condition-weighted. Also we could incorporate overall readiness  $C_t$ ? Not required. The flaw detection uses only windowed  $\Delta E$ , injury, and condition weighting. So  $\phi_p = \alpha * \phi_{\text{base}} + \beta * t_p$ , with  $\alpha + \beta = 1$ ? We can just define  $\phi_p$  as a weighted sum of the two signals, both in  $[-1,1]$ , and then clamp to  $[-1,1]$ . For example:  $\phi_p = \text{clamp}(0.7 * \phi_{\text{base}} + 0.3 * t_p, -1, 1)$ .  $\phi_{\text{base}}$  derived from  $P_p - N_p$  norm.

Now, define  $\phi_{\text{base}}$ :

- Compute  $D_p = (P_p - N_p)$ .
- Normalize by max possible magnitude:  $\text{max\_abs} = E_{\max} * S_{\max}$  (since each term capped at  $E_{\max}$ ). So  $D_{\text{norm}} = D_p / (E_{\max} * S_{\max})$ . That ranges  $[-1,1]$  if all days had max error and weights=1. Then  $\phi_{\text{base}} = \tanh(2 * D_{\text{norm}})$ ? Actually  $D_{\text{norm}}$  is already linear in  $[-1,1]$ ; we can just use  $D_{\text{norm}}$  directly, maybe with a deadband. The tanh might make it smoother but not necessary. Since we want  $\phi_p \in [-1,1]$ ,  $D_{\text{norm}}$  already satisfies that. However, due to attenuation and missing days, it may be smaller. So we can set  $\phi_{\text{base}} = D_{\text{norm}}$  (clamped to  $[-1,1]$ ). But to avoid hunting, we'll apply a deadband: if  $|D_{\text{norm}}| < \delta$ , set to 0. So  $\phi_{\text{base}} = D_{\text{norm}}$  if  $|D_{\text{norm}}| \geq \delta$  else 0.  $\delta = 0.1$  maybe.

Now,  $\phi_p = \text{clamp}(0.7 * \phi_{\text{base}} + 0.3 * t_p, -1, 1)$ . Or we could use more sophisticated blending. But it's fine.

Discrete flaw class: based on  $\phi_p$  thresholds. We define classes: 'capacity\_deficit' for  $\phi_p > \theta_{\text{pos}}$  (e.g., 0.3), 'latent\_headroom' for  $\phi_p < -\theta_{\text{pos}}$  (negative threshold), 'fatigue\_masking' maybe when  $\phi_p$  is near zero but trend is negative and readiness low? We'll include 'fatigue\_masking' if there is a pattern of high  $\Delta E$  but also high readiness? No. The requirement mentions 'fatigue\_masking' as a possible class. I'll define: if  $\phi_p$  is in neutral range but there is a high injury severity or some other signal? Actually, fatigue masking could be when  $\Delta E$  is near zero but readiness is dropping, indicating athlete is pushing through fatigue. That would be derived from  $C_t$ , not just  $\Delta E$ . But F only uses  $\Delta E$ , injury, and condition weighting; we have readiness state vector but not as direct input? The input to F includes  $\text{StateVectorRow}[\leq 21]$ , which includes readiness scores. So we can incorporate readiness. We could detect fatigue masking:  $\phi_p$  neutral but readiness trend declining sharply, and  $\Delta E$  maybe not elevated because athlete is not reporting accurately? That's speculative. Better to stick to simple classes: 'capacity\_deficit', 'latent\_headroom', 'neutral'. And perhaps 'injury\_override' if joint severity high. I'll define three classes: 'capacity\_deficit' ( $\phi_p \geq \theta_{\text{def}}$ ), 'latent\_headroom' ( $\phi_p \leq -\theta_{\text{headroom}}$ ), else 'neutral'. And if any joint severity  $> \text{triageMin}$  for training age, and pattern loads that joint, we might force 'neutral' or a 'caution' class. I'll add 'caution' class when injury present: if max joint severity in window for pattern's joints  $\geq \text{experience.triageMin}$ , then  $\text{flawClass} = \text{'caution'}$  regardless of  $\phi$ ? The requirement: "an injured loaded joint must attenuate a 'train it harder' deficit reading (you

never chase a deficit through a nigggle)." So if there is a nigggle, we should not label as 'latent\_headroom' either (which would lead to increasing load). So we can force 'neutral' or 'caution' when any loaded joint has severity  $\geq$  triageMin. So we can have a rule: if  $l_{p\_severity} \geq$  triageMin,  $flawClass = 'caution'$ , else based on  $\phi$ . That satisfies.

Confidence gating: minimum observations:  $MIN\_OBSERVATIONS = 5$  (e.g., at least 5 days with at least one set of that pattern in window). If count of days with  $n_i > 0 < MIN\_OBSERVATIONS$ , then  $flawClass = 'neutral'$ ,  $\phi_p = 0$ .

Now, output FlawReport: per pattern,  $\phi$  (rounded to 4 decimals? round2 maybe),  $flawClass$ , observations (count of days with data). And window-level summary: maybe overall readiness trend, but we can skip.

Next, control action  $u_{t+1} = g(FlawReport, profile, macroPhase)$ . We must produce per-pattern corrections to apply to the next 4-week block.

The autopilot's job is to adjust the block generator parameters: for each pattern, we can modify prescribed  $target\_rpe$  (the  $rpeWave$  values, or per-slot  $target\_rpe$ ), sets, and load modifier (via daily  $AdjustmentVector$ ). Additionally, we can bias preference (the routing of which patterns get more slots). We need a controller law that maps  $\phi_p$  to adjustments.

Given a flaw class per pattern:

- 'capacity\_deficit': athlete is under-recovering for that pattern ( $\Delta E > 0$ ). So we should reduce intensity (lower  $target\_rpe$ ) and/or reduce volume (sets) to allow recovery, and maybe reduce load. So  $dRpe_p$  negative (reduce effort),  $dSet_p$  negative or zero,  $dLoad_p$  negative (load modifier  $< 1$ ).
- 'latent\_headroom':  $\Delta E < 0$ , athlete can handle more. So we can increase  $target\_rpe$  (toward cap) and/or increase sets, and maybe increase load.  $dRpe_p$  positive,  $dSet_p$  positive,  $dLoad_p > 1$ .
- 'caution': due to nigggle, we should not increase load/effort, but may reduce to protect. So we can set  $dRpe_p \leq 0$ ,  $dSet_p \leq 0$ ,  $dLoad_p \leq 1$ , never positive.
- 'neutral': no change.

The control must be monotone-conservative under guardrails: if any Guardrail is restrictive or nigggle flags a loaded joint,  $u_{t+1}$  may only reduce load/sets/effort for affected patterns — never raise them. So if caution or any restrictive guardrail, we clamp positive adjustments to zero.

We also have bounded authority: total corrective volume added across block  $\leq$  fixed cap, and  $u$  cannot move athlete more than one "step" of load/effort per block. The "step" sizes: load modifier literals are ['0.80', '0.85', '0.90', '0.95', '1.00', '1.05']; one step up/down means moving to adjacent literal. For RPE cap, the literals are 6.5 to 10.0 in 0.5 increments; one step is 0.5. For set modifier, literals -2, -1, 0, 1; one step is  $\pm 1$ . So we'll define allowed per-block adjustment deltas:  $\Delta load \in \{-0.05, 0, +0.05\}$  (if moving from neutral 1.00 to 1.05 or 0.95),  $\Delta rpe\_cap \in \{-0.5, 0, +0.5\}$ ,  $\Delta sets \in \{-1, 0, +1\}$  per pattern. And volume added across patterns limited to, say, total sets added across all patterns  $\leq 2$  per block.

Now, we need to map  $\phi_p$  to these discrete deltas. We'll set thresholds:  $\phi_p > 0.4 \Rightarrow$  strong deficit  $\Rightarrow$  reduce load/effort; between 0.2 and 0.4  $\Rightarrow$  mild deficit; between -0.2 and 0.2  $\Rightarrow$  neutral; between -0.4 and -0.2  $\Rightarrow$  mild headroom;  $< -0.4 \Rightarrow$  strong headroom. Then for mild deficit, we might reduce RPE cap by 0.5 or reduce sets by 1. For strong deficit, reduce load by 0.05 and RPE cap by 0.5, etc. We'll design a deterministic table.

Also, adjustments must respect the existing ceilings ( $base\_rpe\_cap$ , experience caps, flagged ceilings). And the daily  $AdjustmentVector$  derived from block-level corrections will be clamped by  $clampAdjustment$ , which enforces  $load\_modifier \geq 0$  and  $set\_modifier \geq SET\_MODIFIER\_FLOOR$ .

We must incorporate fatigue cost budget: adding sets for headroom must not exceed  $HYBRID\_TAX\_THRESHOLD$  or increase total fatigue cost beyond safe limit. We can check: total added sets across block, weighted by  $SCHEMA\_FATIGUE\_COST$ , must not increase block fatigue cost by more than, say, 10% of base cost. We'll just cap total added sets globally.

For simplicity, we'll define per-pattern corrections:

- $dRpe_p$ : change in  $target\_rpe$  for that pattern's  $rpeWave$  values. But  $rpeWave$  is per week; we can adjust the  $rpeWave$  by adding  $\delta$  to each week's value, clamped to  $[5.0, base\_rpe\_cap]$  and subject to phase mods. We'll store as a  $\delta\_rpe$  for the pattern's base scheme.
- $dSet_p$ : change in number of sets per session for that pattern, relative to block generator's base sets (from SCHEMES and PHASE\_MODS). We'll adjust sets per session by  $dSet_p$  (integer between -2 and +2, but limited to SET\_MODIFIER\_LITERALS later).
- $dLoad_p$ : a load multiplier to apply to working weights for that pattern, snapped to LOAD\_MODIFIER\_LITERALS. This will be a per-pattern  $load\_modifier$  that overrides the daily  $AdjustmentVector.load\_modifier$  when that pattern is performed? The  $AdjustmentVector$  is session-global (a single  $load\_modifier$ ,  $set\_modifier$ ,  $rpe\_cap$  for the whole day). But the requirement says  $u_{\{t+1\}}$  must degrade to the existing  $AdjustmentVector$  domain at the daily grain. That means the control action must produce a daily  $AdjustmentVector$  that applies to all movements on that day, not per-pattern specific? But the problem statement: "A per-pattern correction  $u_p = \{ dLoad_p, dSet_p, dRpe_p, prefBias_p \}$  where ...  $dLoad_p$  is snapped onto LOAD\_MODIFIER\_LITERALS when surfaced as a daily  $AdjustmentVector.load\_modifier$ ." This suggests that the daily  $AdjustmentVector$ 's  $load\_modifier$  is a single value for the session, but we can have per-pattern corrections that get merged into a session-level vector. That seems contradictory: if you have different load modifiers for different patterns, you can't express that with a single global  $load\_modifier$ . Unless the system applies per-pattern overrides via the slot plan's applied\_prefixes or something else. The existing  $AdjustmentVector$  is session-level. The control action may produce a block plan that includes per-pattern  $target\_rpe$  and sets adjustments, and for load, we might modify the planned working weight via a per-pattern load multiplier that gets translated into a daily  $load\_modifier$  when only that pattern is trained? But a day can have multiple patterns. So the  $load\_modifier$  must apply to all exercises equally. That's a limitation. The problem might be that the autopilot outputs a session-level  $AdjustmentVector$ , but we want per-pattern logic. We'll interpret that the control action yields a set of per-pattern corrections that are then aggregated into a single daily  $AdjustmentVector$  by choosing the most conservative (or some priority) for that day's patterns. But the requirement: " $u_{\{t+1\}} = g(FlawReport, profile, macroPhase)$  that auto-corrects the next block. It must be expressible as a perturbation of the §3.9 generator tables and must degrade to the existing  $AdjustmentVector$  domain at the daily grain." So the control action produces a block-level plan (perturbations to the generator tables), and for each day, a daily  $AdjustmentVector$  is derived from that plan. The daily  $AdjustmentVector$  has a single  $load\_modifier$ ,  $set\_modifier$ ,  $rpe\_cap$ . So if the block plan has per-pattern load modifiers, we'd need to average or pick the most restrictive for that day's workout. I'll assume that each day's workout contains a subset of patterns; the daily  $AdjustmentVector$  will be set based on the pattern(s) scheduled for that day, taking the most conservative (lowest  $load\_modifier$ , lowest  $set\_modifier$ , lowest  $rpe\_cap$ ) among the patterns exercised that day, to ensure safety. That seems plausible.

Thus, per-pattern  $u_p = \{ dLoad_p, dSet_p, dRpe_p, prefBias_p \}$ . We'll map  $\phi_p$  to these using gain tables.

Now, define controller law.

First, determine base scheme parameters for the pattern from current profile: objective, training age, etc. The block generator already uses SCHEMES and PHASE\_MODS. We'll compute  $base\_rpe\_wave$  for the pattern (e.g., from SCHEMES[objective]. $rpeWave$ , adjusted by PHASE\_MODS[macroPhase]. $rpe$ ). That gives  $target\_rpe$  for weeks 1-3. We'll apply  $dRpe_p$  as a  $\delta$  to each week's value, but clamped to  $[5.0, base\_rpe\_cap]$  where  $base\_rpe\_cap$  is from profile (experience-based cap and any guardrails). So new  $target\_rpe = clamp(original\_target\_rpe + dRpe_p, 5.0, cap)$ .  $dRpe_p$  can be -0.5, 0, +0.5.

$dSet_p$ : change number of sets per session for that pattern. Base sets from SCHEMES and PHASE\_MODS.  $dSet_p \in \{-1, 0, 1\}$  (maybe -2 if strong deficit?). We'll limit to  $\pm 1$  per block to stay bounded.



dLoad\_p: a multiplier in {0.95, 0.90, 0.85, 0.80, 1.00, 1.05}? But LOAD\_MODIFIER\_LITERALS has 0.80,0.85,0.90,0.95,1.00,1.05. So dLoad\_p would be a delta from 1.0, but we need to snap to those literals. So we'll choose a target load multiplier value based on deficit/headroom and current load modifier from last block? To be anti-windup, we limit change per block to one step. So we'll compute desired load modifier  $L_{des} = \text{current\_load\_modifier}$  (from previous block's daily load\_modifier average?) But we don't have memory of previous block's modifier; but we can assume the block generator starts from 1.00 each block unless overridden. So we can define  $L_{des} = 1.00 + \Delta L$ , where  $\Delta L$  is chosen from  $\{-0.05, 0, +0.05\}$  based on  $\phi$ . Then we snap to nearest literal: we can map to literals by finding closest value in array. Because we round to 2 decimals (since load\_modifier is number, not necessarily exact literal due to rounding? But it should be one of the literals after snapping). We'll define a helper to snap to nearest literal.

prefBias\_p: -1,0,1 to bias substitution router. If deficit, bias negative to avoid pattern; if headroom, bias positive to prioritize. So we set based on  $\phi$ : deficit  $\rightarrow -1$ , headroom  $\rightarrow +1$ , neutral  $\rightarrow 0$ .

Now, safety invariants:

- If any guardrail restricts ( $\text{load\_multiplier} < 1$ ,  $\text{set\_delta} < 0$ ,  $\text{rpe\_cap\_max} < 10$ , or halt), we must apply reductions: for any positive adjustments (headroom), we force them to zero or negative? Monotone-conservative: "if any Guardrail is restrictive or  $I_t$  flags a loaded joint,  $u_{t+1}$  may only reduce load/sets/effort for affected patterns — never raise them." So if guardrail restrictive for a pattern (maybe because of niggles in that pattern's joints), then  $\text{dLoad}_p \leq 1$ ,  $\text{dSet}_p \leq 0$ ,  $\text{dRpe}_p \leq 0$ ,  $\text{prefBias}_p \leq 0$ . We'll enforce that for patterns with loaded joints that have severity  $\geq \text{triageMin}$  or if any guardrail applies globally? Guardrails might be per pattern? The Guardrail interface doesn't specify pattern; it might be global. But the invariant says "for affected patterns". We'll assume that if there is any restrictive guardrail ( $\text{load\_multiplier} < 1$ , or  $\text{set\_delta} < 0$ , or  $\text{rpe\_cap\_max} < 10$ ), then all patterns are restricted? Actually the guardrail could be specific to a pattern if it's triggered by an injury in that pattern's joints. But we'll generalize: any pattern that involves a joint with severity  $\geq \text{triageMin}$  is considered "affected" and gets only reductions. Similarly, if a global guardrail exists (maybe due to acute fatigue), then all patterns might be limited. So we'll treat the guardrail's load\_multiplier as a cap for load\_modifier, rpe\_cap\_max as cap for rpe, set\_delta as cap for set modifications (negative delta means we can only reduce sets). We'll incorporate that when computing adjustments.
- Halt supremacy: if halt is true, we don't produce any positive adjustments, and maybe we force all corrections to neutral or negative? But halt means hard stop — likely the block is paused. We'll set all  $\text{dLoad}_p = 1$ ,  $\text{dSet}_p = 0$ ,  $\text{dRpe}_p = 0$ ,  $\text{prefBias} = 0$  (no changes). Actually halt may be per pattern? We'll treat halt globally: if any halt flag, we return no adjustments (neutral). So  $u_{t+1}$  will be identity.
- Volume bifurcation: corrections reason over effective tonnage where relevant but never rewrite raw mech\_daily/ACWR pipeline. That means we won't modify raw volume tracking; we only adjust prescription (target\_rpe, sets, load modifier) which affect future effective load, but the raw tonnage will still be computed from actual reps and load. That's fine.

Now, design the mapping from  $\phi_p$  to correction deltas.

We'll define a deadband:  $|\phi_p| < 0.15 \rightarrow \text{no action}$ . Then:

- $0.15 \leq \phi_p < 0.4$ : mild deficit: reduce target\_rpe by 0.5, reduce sets by 1 if possible (but not below min sets), load\_modifier unchanged (or maybe reduce to 0.95 if needed? Keep load same to avoid strength loss, but reduce effort and volume).  $\text{prefbias} = -1$ .
- $\phi_p \geq 0.4$ : strong deficit: reduce load\_modifier one step (0.95 if currently 1.0), reduce target\_rpe by 0.5, reduce sets by 1 (maybe 2 if many sets?),  $\text{prefbias} = -1$ .
- $-0.4 < \phi_p \leq -0.15$ : mild headroom: increase target\_rpe by 0.5 (capped), increase sets by 1 (if within total budget), load\_modifier unchanged (or increase to 1.05?),  $\text{prefbias} = +1$ .
- $\phi_p \leq -0.4$ : strong headroom: increase load\_modifier one step (1.05), increase target\_rpe by 0.5, increase sets by 1,  $\text{prefbias} = +1$ .



But we must respect the bounded authority per block: total set additions across all patterns  $\leq 2$ , total set reductions no limit? Volume reduction is fine. Also, load modifier step change only one level per block.

We also need to consider the current macroPhase: if in deload week (week 4), maybe we don't apply positive adjustments (headroom) because deload is for recovery; but the block corrections apply to the whole block, not just deload week. The block is 4 weeks; weeks 1-3 are accumulation/intensification/realization, week 4 is deload. The control action corrects the next 4-week block, presumably before it's generated. So we apply adjustments to the whole block, but the deload week has fixed reduced load (maybe 50% or something) by the generator. The autopilot might override that. We'll assume the generator already applies deload reduction, and our per-pattern  $dLoad_p$ ,  $dSet_p$ ,  $dRpe_p$  will be applied on top of that. But we should ensure that positive adjustments during deload are not applied. We'll check if macroPhase is 'deload'? The macroPhase argument is the current phase for the upcoming block? Actually, the block structure has phases: gpp, hypertrophy, volume, peak. Deload is a week, not a phase. The macroPhase corresponds to the training block phase. I think we can ignore special deload handling; just apply corrections to the base scheme.

Now, compute the daily AdjustmentVector from the block corrections. Since the daily workout may have multiple patterns, we'll take the most conservative (lowest load\_modifier, lowest set\_modifier, lowest rpe\_cap) among patterns scheduled that day. The set\_modifier is a delta on sets per movement, but the daily AdjustmentVector.set\_modifier is a global delta applied to all movements' planned sets? The existing AdjustmentVector.set\_modifier is an integer delta (e.g., -1) that modifies the number of working sets per movement for the session. That aligns with our per-pattern  $dSet_p$ . But if different patterns have different  $dSet_p$ , we need a single set\_modifier. We could take the minimum (most negative) across all patterns scheduled that day, to be safe. That would ensure we don't over-prescribe. Similarly, rpe\_cap: we'd take the minimum (lowest ceiling) across patterns, because a lower cap restricts all movements. load\_modifier: we'd take the minimum (lowest multiplier) across patterns, as reducing load is safer. So we'll aggregate by taking element-wise min (for load\_modifier, rpe\_cap) and min for set\_modifier (since negative is reduction).

But then a pattern with latent headroom would get a positive  $dSet_p$ , but if another pattern on the same day has deficit, the daily set\_modifier would be negative, overriding the headroom gain. That may be too conservative. However, the autopilot's per-pattern corrections are expressed in the block plan as adjustments to the set counts per movement. The daily AdjustmentVector could be derived from the specific pattern being performed, or we could compute a session vector that takes the average? The problem says " $u_{t+1}$  must degrade to the existing AdjustmentVector domain at the daily grain." So we must produce a function that yields an AdjustmentVector for a given day based on the block plan and the patterns scheduled that day. I'll define a function

```
deriveDailyAdjustment(blockCorrections: ControlAction, patterns: MovementPattern[]):
```

```
AdjustmentVector
```

that takes the set of patterns exercised that day and computes the daily adjustment as the most conservative (min) across those patterns. That is deterministic and safe.

Now, constant tables.

We'll need:

- $\lambda$  (decay factor for EMA): e.g., 0.9? 0.85? We'll pick 0.88 (so that last day weight  $\sim 1$ , day-20 weight  $\lambda^{20} \sim 0.88^{20} \approx 0.1$ ). That gives reasonable window.
- $E_{max} = 3.0$
- $J_{max} = 10$
- Gain for  $\phi_{base}$  normalization: we'll just use  $D_{norm}$  as is, so no gain.
- $\phi$  deadband:  $\delta = 0.15$
- Weights for combining  $\phi_{base}$  and  $t_p$ :  $w_{base} = 0.7$ ,  $w_{trend} = 0.3$
- Thresholds for flaw classification:  $\theta_{def} = 0.3$ ,  $\theta_{headroom} = 0.3$  (since  $\phi_p > 0.3 \Rightarrow$  deficit,  $< -0.3 \Rightarrow$  headroom)

- MIN\_OBSERVATIONS = 5
- For control mapping: deadband for  $\phi_p$ : 0.15; mild thresholds: 0.15 to 0.4; strong:  $\geq 0.4$ ; similarly negative.
- Set addition budget: MAX\_ADDED\_SETS = 2
- Load modifier step: LOAD\_STEP = 0.05 (one literal step)
- RPE step = 0.5
- Set step = 1
- Default load modifier literals array.

We also need to incorporate experience triage for load scaling and cap delta. That can affect the allowed rpe cap and load adjustments. For instance, for beginners, loadScale = 1.2 means we might want to reduce load more aggressively? The existing EXPERIENCE\_TRIAGE has loadScale (for load modifier?), capDelta (added to rpe cap?), extraSetCut (sets reduction). I think these are used in the existing AdjustmentVector generation, not in the autopilot's block corrections. The autopilot's corrections may interact with them. We'll incorporate: when determining max allowed rpe cap, we take base\_rpe\_cap from profile (which may be lowered by experience). Also, when computing load modifier, we could scale the step by loadScale? The requirement says we should use existing EXPERIENCE\_TRIAGE. We'll include a rule: for beginners, any load increase is forbidden? Not necessarily, but the experience tables already give conservative values. We'll just apply the profile's base\_rpe\_cap (which already factors in experience) when clamping rpe adjustments. For load, we might limit increase for beginners by not allowing load\_modifier > 1.0? I'll incorporate: if trainingAge is 'beginner', then dLoad\_p max = 1.0 (no increase). Similar for capDelta? capDelta is -2.0 for beginner, meaning rpe cap is lowered by 2.0 relative to base. So rpe cap max = (base\_rpe\_cap - 2.0) perhaps? We'll use base\_rpe\_cap as already computed. So the autopilot's rpe cap will be min(10, base\_rpe\_cap + dRpe\_p) but dRpe\_p may not exceed the cap delta. We'll set a max allowed target\_rpe = base\_rpe\_cap (from profile), which already includes experience reductions. So that's enough.

Now, design the closed-form recurrence for the EMA sums.

We'll compute over the array of daily observations for a pattern. We have arrays e\_arr, w\_arr, n\_arr, j\_arr for  $i=0..W-1$  ( $W=21$ ). Order:  $i=0$  oldest,  $i=W-1$  most recent. We'll compute:

$$S_{\max} = (1 - \lambda^W) / (1 - \lambda) \quad (\text{if } \lambda \neq 1; \lambda=0.88).$$

For each  $i$ , if  $n_i > 0$ :

$$\text{weight}_i = \lambda^{(W-1-i)}$$

$$w_{\text{att}_i} = w_{\text{arr}}[i] \quad (\text{already computed average attenuation per day})$$

$$e_{\text{clamped}} = \text{clamp}(e_{\text{arr}}[i], -E_{\max}, E_{\max})$$

$$j_i = j_{\text{arr}}[i]$$

if  $e_{\text{clamped}} > 0$ :

$$P += \text{weight}_i * w_{\text{att}_i} * e_{\text{clamped}} * \max(0, 1 - j_i/J_{\max})$$

else if  $e_{\text{clamped}} < 0$ :

$$N += \text{weight}_i * w_{\text{att}_i} * (-e_{\text{clamped}}) \quad // \text{positive for headroom accumulation}$$

total\_weight += weight\_i // for observation count? Actually we need count of days with observations for gating. We'll simply count days with  $n_i > 0$ : obs\_count = number of  $i$  where  $n_i > 0$ .

Then  $D = (P - N) // P$  is positive accumulation,  $N$  is positive headroom magnitude.

$$D_{\text{norm}} = D / (E_{\max} * S_{\max}) \quad // \text{clamped to } [-1,1] \text{ already? Since max possible } D \text{ is } E_{\max} *$$

$S_{\max}$  (if all  $w_{\text{att}}=1$ , no injury), so  $D_{\text{norm}} \in [-1,1]$  theoretically. But due to attenuation, it may be lower.

Apply deadband: if  $\text{abs}(D_{\text{norm}}) < \delta$ ,  $D_{\text{norm}} = 0$ .

$$\phi_{\text{base}} = D_{\text{norm}}.$$

Trend: compute recent mean and older mean. For days  $i$  in  $[W-7, W-1]$  (last 7 days) and  $i$  in  $[0,6]$  (first 7 days). For each subset, compute average of  $e_i$  over days with  $n_i > 0$ ; if no observations, mean = 0. Then trend\_raw = mean\_recent - mean\_old.  $t_p = \tanh(\text{trend\_raw} / T_{\text{scale}})$ ,  $T_{\text{scale}} = 1.0$ . So  $t_p \in (-1,1)$ . Deterministic.

Then  $\phi_p = w_{\text{base}} * \phi_{\text{base}} + w_{\text{trend}} * t_p$ , clamped to  $[-1,1]$ . Round to round4.

Flaw classification: if  $\text{obs\_count} < \text{MIN\_OBSERVATIONS}$ :  $\text{flawClass} = \text{'neutral'}$ . Else if  $\max(j_i \text{ over window}) \geq \text{triageMin}$  (from  $\text{EXPERIENCE\_SEVERITY}[\text{trainingAge}].\text{triageMin}$ ):  $\text{flawClass} = \text{'caution'}$ . Else if  $\phi_p \geq \theta_{\text{def}}$ :  $\text{'capacity\_deficit'}$ . Else if  $\phi_p \leq -\theta_{\text{headroom}}$ :  $\text{'latent\_headroom'}$ . Else  $\text{'neutral'}$ .

Note: triageMin depends on trainingAge, which is in user profile, not in window. So we need profile input to F? The F signature only gets windowed history; it doesn't get profile. But we need trainingAge for triage thresholds. The F map originally said:  $\{ \text{StateVectorRow}[\leq 21], \text{perPattern}\Delta E[\leq 21], \text{niggleRows}[\text{window}] \} \rightarrow \text{FlawReport}$ . It doesn't include profile. So we could hardcode thresholds that are universal, not experience-dependent? The requirement says "The operative/halt thresholds are experience-weighted via  $\text{EXPERIENCE\_SEVERITY}[\text{trainingAge}]$ ". So we might need to pass trainingAge into F. I'll add trainingAge as an extra parameter to F, as it's part of athlete profile that is static. Or we could incorporate the triage check later in control, but F uses it for flaw class. We'll add trainingAge to F inputs. In the final signatures, I'll include it.

Now control action.

Inputs: FlawReport, UserProfile (with trainingAge, base\_rpe\_cap, maybe current block's load modifier? But we don't have memory of previous block's adjustments; we assume the block generator starts from neutral settings. So we'll consider the baseline as 1.0 load, 0 set delta, rpe cap from profile).

Compute for each pattern p:

- base\_target\_rpe from  $\text{SCHEMES}[\text{objective}].\text{rpeWave}$  and  $\text{PHASE\_MODS}[\text{macroPhase}].\text{rpe}$ ? We need objective and macroPhase. These are part of profile/current block. So we need them as input. We'll include objective and macroPhase in profile. Then we can compute base\_rpe for weeks 1-3: for each week  $w$  (0,1,2),  $\text{rpe\_wave}[w] + \text{phaseMod.rpe}$ . For simplicity, we'll assume the autopilot adjusts the average target\_rpe over the accumulation weeks. We'll define the correction as  $\text{delta\_rpe}$  to add to the base rpe values for that pattern, clamped.

So  $u_p.\text{dRpe}_p$  = determined from  $\phi_p$  and caution flags, clamped to allowed step  $\pm 0.5$ .

$u_p.\text{dSet}_p$  = similar, integer in  $\{-1,0,1\}$  (or -2 if strong deficit and base sets  $>1$ ). We'll limit to  $\pm 1$  for headroom, but for deficit we could allow -2 only if base sets  $\geq 3$ ? We'll just use -1 as max reduction per block to be conservative. Total set reductions are not limited by budget.

$u_p.\text{dLoad}_p$ : determine target load multiplier  $L_{\text{target}}$ . Start from neutral 1.00. If strong deficit:  $L_{\text{target}} = 0.95$  (one step down from 1.00). If strong headroom:  $L_{\text{target}} = 1.05$ . If mild, keep 1.00. Then snap to nearest literal in  $\text{LOAD\_MODIFIER\_LITERALS}$ . For 1.00, it's there. So  $\text{dLoad}_p$  is the target value, not delta. We'll store as a load\_modifier value.

$u_p.\text{prefBias}_p$ : as described.

Now, enforce monotone-conservative under flags:

- For each pattern  $p$ , if any of its loaded joints ( $\text{PATTERN\_JOINTS}[p]$ ) has a recent niggle with severity  $\geq \text{triageMin}$  (from profile), then:  
 $\text{dRpe}_p = \min(0, \text{dRpe}_p)$ ;  $\text{dSet}_p = \min(0, \text{dSet}_p)$ ;  $\text{dLoad}_p = \min(1.0, \text{dLoad}_p)$ ;  $\text{prefBias}_p = \min(0, \text{prefBias}_p)$ .
- Also, if there is an active Guardrail (we need guardrail input to control action), perhaps from recent restrictive report. The control function will receive current guardrails (maybe as part of FlawReport or profile). We'll assume FlawReport includes any active guardrails (halt flag). I'll add a field in FlawReport:  $\text{globalGuardrail: Guardrail} \mid \text{null}$ . If guardrail exists and restrictive, then for all patterns, apply the same limiting:  $\text{dRpe}_p \leq 0$ ,  $\text{dSet}_p \leq 0$ ,  $\text{dLoad}_p \leq 1$ ,  $\text{prefBias}_p \leq 0$ . And if halt is true, then all deltas = neutral.
- Bounded set additions: total positive  $\text{dSet}_p$  across all patterns (where  $\text{dSet}_p > 0$ ) must not exceed  $\text{MAX\_ADDED\_SETS} = 2$ . If sum exceeds, prioritize patterns with most negative  $\phi_p$  (strongest headroom) and reduce others to 0. We'll sort by  $\phi_p$  ascending and assign +1 to top 2.

- Clamp final dLoad\_p to LOAD\_MODIFIER\_LITERALS (we already snap). Also ensure load\_modifier  $\geq 0$  (always).
- dRpe\_p step must be such that final target\_rpe per pattern, per week, remains in [5.0, base\_rpe\_cap] (base\_rpe\_cap from profile, already incorporating experience and flags). So we'll clamp after adding delta.

Now, produce ControlAction interface with per-pattern corrections and block-level summary (total added sets, etc.). Also a function to derive daily AdjustmentVector: given day's patterns, compute session\_load\_modifier = min over patterns of dLoad\_p (if dLoad\_p  $< 1$ , take min; if  $> 1$ , could we allow higher? For safety, we'll take the min of dLoad\_p, because load\_modifier applies to all exercises. If a pattern has headroom (dLoad\_p 1.05) and another has deficit (0.95), the daily load\_modifier should be 0.95 to avoid overloading the deficit pattern. So min is safe. Similarly, set\_modifier = min over patterns' dSet\_p (since negative reduces sets). rpe\_cap = min over patterns' (base\_rpe\_cap + dRpe\_p?), but dRpe\_p adjusts target\_rpe, not rpe cap. The AdjustmentVector has rpe\_cap, a hard ceiling for the session. We could set rpe\_cap = min over patterns of (base\_rpe\_cap + dRpe\_p) but that would be per pattern ceiling. Since the session cap is global, we'd take the minimum cap among patterns scheduled, to ensure no pattern exceeds its allowed effort. So session rpe\_cap = min( base\_rpe\_cap + dRpe\_p for all p in day ) capped to [5,10] with 0.5 rounding. And the coaching\_cue we can generate a deterministic string based on most severe flaw.

This seems solid.

Now, invariants list:

1. Determinism: all operations use fixed arithmetic, no randomness, fixed rounding rules, ensuring byte-identical output given same inputs.
2. Monotone-conservative under flags: if any guardrail restrictive or injury severity  $\geq$  triageMin for a joint loaded by pattern,  $u_{\{t+1\}}$  only reduces load/sets/effort for that pattern.
3. Domain closure: daily AdjustmentVector derived from ControlAction satisfies load\_modifier  $\in$  LOAD\_MODIFIER\_LITERALS  $\cup \{1.0?$  actually literals include 1.00}, set\_modifier  $\in$  SET\_MODIFIER\_LITERALS, rpe\_cap  $\in$  RPE\_CAP\_LITERALS after clampAdjustment.
4. Halt supremacy: if any halt flag is true, all corrections neutral.
5. Bounded authority per block: total added sets  $\leq$  MAX\_ADDED\_SETS, load modifier change  $\leq$  0.05, rpe change  $\leq$  0.5, set change per pattern  $\leq$  1 (except possible -2 for deficit but limited to -1 to be safe). We'll state max per pattern set change  $\pm 1$ , total positive sets  $\leq$  2.
6. Volume bifurcation: effective tonnage used for condition weighting but raw tonnage/ACWR pipeline untouched.

Now, write the closed-form formulas for F in the final output. We'll present them as math definitions.

We'll need to define the computation of per-pattern daily  $\Delta E$  and effective load ratio. We'll assume access to set\_record rows with columns: pattern, reps, load\_kg, rpe\_actual, slot\_target\_rpe, effective\_load\_kg, base\_load\_kg. We'll compute  $e = rpe\_actual - slot\_target\_rpe$ , and effectiveLoadRatio = effective\_load\_kg / base\_load\_kg if base\_load\_kg  $> 0$  else 1. Then for each day, group by pattern.

For each pattern p, we produce arrays over the 21-day window (days t-20 to t). Let the days be indexed d=0..20, where 0 is oldest. For each day d:

- Let  $S_d$  be the set of sets for pattern p on that day. If empty,  $n_d=0$ .
- If  $n_d > 0$ :  

$$e_d = (1/n_d) * \sum_{s \in S_d} (rpe\_actual\_s - target\_rpe\_s)$$

$$w_d = (1/n_d) * \sum_{s \in S_d} (1 / \max(1, effectiveLoadRatio\_s))$$
// using effectiveLoadRatio = effective\_load / base\_load
- Joint severity  $j_d = \max\{niggle.severity \mid niggle.region \in PATTERN\_JOINTS[p] \text{ and } niggle.reported\_at\_ms \text{ falls on day } d\}$ . If none, 0.

Then compute P, N, S\_max as formulas.

$$P = \sum_{d: e_d > 0} \lambda^{\{(20-d)\}} * w_d * \min(e_d, E_{\max}) * \max(0, 1 - j_d/10)$$

$$N = \sum_{d: e_d < 0} \lambda^{\{(20-d)\}} * w_d * \min(|e_d|, E_{\max}) \text{ (note: we don't attenuate N with injury, but we'll not include } j_d \text{ here)}$$

$$S_{\max} = (1 - \lambda^{21}) / (1 - \lambda)$$

$$D_{\text{norm}} = (P - N) / (E_{\max} * S_{\max})$$

If  $|D_{\text{norm}}| < \delta$ :  $D_{\text{norm}} = 0$ .

$$\phi_{\text{base}} = D_{\text{norm}}.$$

Trend:  $e_{\text{old}}$  = average of  $e_d$  over  $d=0..6$  where  $n_d > 0$ , else 0.  $e_{\text{recent}}$  = average over  $d=14..20$

where  $n_d > 0$ , else 0.  $\text{trend}_{\text{raw}} = e_{\text{recent}} - e_{\text{old}}$ .  $t_p = (\exp(2 * \text{trend}_{\text{raw}}) - 1) /$

$(\exp(2 * \text{trend}_{\text{raw}}) + 1) // \tanh$

$$\phi_p = \text{clamp}(0.7 * \phi_{\text{base}} + 0.3 * t_p, -1, 1). \text{ Round to 4 decimals.}$$

Observations count  $\text{obs}_p$  = number of  $d$  with  $n_d > 0$ .

Flaw class: if  $\text{obs}_p < \text{MIN\_OBSERVATIONS} \Rightarrow$  'neutral'; else if  $\max(j_d \text{ over } d) \geq \text{triageMin} \Rightarrow$  'caution'; else if  $\phi_p \geq \theta_{\text{def}} \Rightarrow$  'capacity\_deficit'; else if  $\phi_p \leq -\theta_{\text{headroom}} \Rightarrow$  'latent\_headroom'; else 'neutral'.

Now control action.

Compute for each pattern  $p$ :

- Determine  $\text{base\_rpe}$  for pattern from  $\text{SCHEMES}[\text{objective}].\text{rpeWave}$  and  $\text{PHASE\_MODS}[\text{macroPhase}].\text{rpe}$ . We'll take the average of the three wave values plus phase mod, as a representative  $\text{target\_rpe}$  for the pattern. Then desired delta based on  $\phi_p$  and  $\text{flawClass}$ .

If  $\text{flawClass} ==$  'caution' or  $\text{globalGuardrail.restrictive}$  (i.e.,  $\text{load\_multiplier} < 1$  or  $\text{set\_delta} < 0$  or  $\text{rpe\_cap\_max} < 10$ ) or halt:

$\text{dRpe}_p = \min(0, \delta_{\text{rpe\_based\_on\_}\phi})$  but clamped to 0 if positive. Similarly  $\text{dSet}_p = \min(0, \dots)$ ,  $\text{dLoad}_p = \min(1.0, \dots)$ .

Else:

Use  $\phi_p$  thresholds:

if  $\phi_p \geq 0.4$ : strong deficit  $\rightarrow \text{dRpe} = -0.5, \text{dSet} = -1, \text{dLoad\_target} = 0.95, \text{prefBias} = -1$

elif  $\phi_p \geq 0.15$ : mild deficit  $\rightarrow \text{dRpe} = -0.5, \text{dSet} = 0$  (or -1? I'll set  $\text{dSet} = -1$  for mild too to reduce volume),  $\text{dLoad} = 1.0, \text{prefBias} = -1$

elif  $\phi_p \leq -0.4$ : strong headroom  $\rightarrow \text{dRpe} = +0.5, \text{dSet} = +1, \text{dLoad\_target} = 1.05, \text{prefBias} = +1$

elif  $\phi_p \leq -0.15$ : mild headroom  $\rightarrow \text{dRpe} = +0.5, \text{dSet} = +1, \text{dLoad} = 1.0, \text{prefBias} = +1$

else: neutral  $\rightarrow$  all zero.

But then we must clamp  $\text{dLoad\_target}$  to literals and ensure one step from 1.0. So 0.95 and 1.05 are valid steps.

After applying safety limits, we set  $\text{dLoad}_p = \text{snapToLiteral}(\text{dLoad\_target})$ .  $\text{snapToLiteral}$  finds closest literal in  $\text{LOAD\_MODIFIER\_LITERALS}$ .

Now, total set additions: sum of  $\text{dSet}_p$  over  $p$  where  $\text{dSet}_p > 0$ . If  $> \text{MAX\_ADDED\_SETS}$ , sort by  $\phi_p$  ascending (most negative first) and keep only first  $\text{MAX\_ADDED\_SETS}$  positive  $\text{dSet}_p$ , set rest to 0.

Now, produce  $\text{ControlAction}$ : record per pattern.

Then daily  $\text{AdjustmentVector}$ : given a day's patterns  $P_{\text{day}}$ , compute:

$\text{load\_mod} = \min_{p \text{ in } P_{\text{day}}} \text{dLoad}_p$  (if  $\text{dLoad}_p$  not present, 1.0)

$\text{set\_mod} = \min_{p \text{ in } P_{\text{day}}} \text{dSet}_p$

$\text{rpe\_cap} = \min_{p \text{ in } P_{\text{day}}} (\text{clamp}(\text{base\_rpe\_cap} + \text{dRpe}_p, 5.0, 10.0))$  and round to nearest 0.5, and ensure is in  $\text{RPE\_CAP\_LITERALS}$  by snapping (pick closest literal).

$\text{coaching\_cue}$ : generate from the most extreme  $\phi_p$  (largest absolute) among patterns with non-

neutral flaw. We'll just output a deterministic sentence like "Reduce effort on [pattern] due to capacity deficit" etc. Can be simple.

We'll need base\_rpe\_cap from profile: existing base\_rpe\_cap  $\in [5.0, 10.0]$  in 0.5 steps. So rpe\_cap derived will be in that range.

Finally, ensure domain closure: load\_modifier will be one of the literals after snapping; set\_modifier integer between -2 and 1, but we limit to -1.1, so it's in SET\_MODIFIER\_LITERALS; rpe\_cap in RPE\_CAP\_LITERALS after rounding and snapping.

Now, constant tables. We'll define all constants used.

Let's pick numeric values:

$$\lambda = 0.88$$

$$E_{\max} = 3.0$$

$$\delta = 0.15$$

$$\theta_{\text{def}} = 0.3, \theta_{\text{headroom}} = 0.3 \text{ (same)}$$

$$\text{MIN\_OBSERVATIONS} = 5$$

$$w_{\text{base}} = 0.7, w_{\text{trend}} = 0.3$$

$$T_{\text{scale}} = 1.0 \text{ (for trend tanh)}$$

$$\text{MAX\_ADDED\_SETS} = 2$$

$$\text{LOAD\_STEP} = 0.05 \text{ (implicit)}$$

LOAD\_MODIFIER\_LITERALS array already defined.

Also, we need to define the snapToLiteral function: find literal with minimal absolute difference; if tie, choose the lower (conservative). Since literals are sorted, we can do a simple binary search.

Now, invariants.

Proceed to write final output.

<thinking> We need to design the flaw detector  $F$  and controller  $u_{\{t+1\}}$  under all constraints. Let's derive closed-form math.

First, model the per-pattern daily  $\Delta E$  and condition weighting.

For each movement pattern  $p$ , over the trailing 21-day window (days  $t-20$  through  $t$ , inclusive), we collect all set records. For each day  $d$  ( $0$  = oldest,  $20$  = most recent), define:

- $n_p(d)$  = number of sets logged for pattern  $p$  on day  $d$ .
- If  $n_p(d) > 0$ :
  - $e_p(d) = (1 / n_p(d)) * \sum_{\text{set}} (\text{rpe\_actual} - \text{target\_rpe})$  ( $\Delta E$ , signed)
  - $\text{eff\_ratio}(\text{set}) = \text{effective\_load\_kg} / \text{base\_load\_kg}$  if  $\text{base\_load\_kg} > 0$  else  $1$
  - $w_p(d) = (1 / n_p(d)) * \sum_{\text{set}} 1 / \max(1, \text{eff\_ratio}(\text{set}))$  (attenuation weight)
  - else:  $e_p(d) = \text{null}$ ,  $w_p(d) = 0$ .
- $j_p(d) = \max \text{ severity among niggle rows with region } \in \text{PATTERN\_JOINTS}[p] \text{ on day } d, \text{ or } 0 \text{ if none.}$

We'll compute  $\phi_p$  in several steps.

## F – Flaw Detection closed-form

1. Exponential decay weights:

$\lambda \in (0, 1)$  fixed constant. For day index  $i=0..20$ , weight  $\omega_i = \lambda^{(20-i)}$ .

$$S_{\max} = \sum_{i=0}^{20} \lambda^i = (1 - \lambda^{21}) / (1 - \lambda).$$

## 2. Accumulation:

$$P_p = \sum_{i=0}^{20} [\text{if } e_p(i) > 0] \omega_i * w_p(i) * \min(e_p(i), E_{\max}) * \max(0, 1 - j_p(i)/J_{\max})$$

$$N_p = \sum_{i=0}^{20} [\text{if } e_p(i) < 0] \omega_i * w_p(i) * \min(|e_p(i)|, E_{\max})$$

( $N_p$  is positive magnitude of headroom; no injury attenuation for headroom.)

## 3. Normalized base signal:

$$D_p = P_p - N_p$$

$$D_{\text{norm}} = D_p / (E_{\max} * S_{\max}) \text{ (ranges roughly } [-1,1])$$

Apply deadband: if  $|D_{\text{norm}}| < \delta \rightarrow D_{\text{norm}} := 0$ .

$$\phi_{\text{base}} = D_{\text{norm}}$$

## 4. Trend:

Compute mean  $e$  over oldest 7 days ( $i=0..6$ , where  $n>0$ ) as  $e_{\text{old}}$  (if none, 0).

Compute mean  $e$  over newest 7 days ( $i=14..20$ , where  $n>0$ ) as  $e_{\text{recent}}$  (if none, 0).

$$\text{trend}_{\text{raw}} = e_{\text{recent}} - e_{\text{old}}$$

$$t_p = \tanh(\text{trend}_{\text{raw}} / T_{\text{scale}}) \text{ // } \tanh(x) = (\exp(2x)-1)/(\exp(2x)+1)$$

## 5. Combine:

$$\phi_p = \text{clamp}(w_{\text{base}} * \phi_{\text{base}} + w_{\text{trend}} * t_p, -1, 1)$$

Round to 4 decimal places:  $\text{round4}(\phi_p)$ .

## 6. Observations count:

$$\text{obs}_p = \text{number of days in window with } n_p(i) > 0.$$

## 7. Flaw classification:

If  $\text{obs}_p < \text{MIN\_OBSERVATIONS} \rightarrow \text{flawClass} = \text{'neutral'}$

Else if  $\max_i j_p(i) \geq \text{triageMin}$  (from  $\text{EXPERIENCE\_SEVERITY}[\text{trainingAge}]$ )  $\rightarrow$

$$\text{flawClass} = \text{'caution'}$$

Else if  $\phi_p \geq \theta_{\text{def}} \rightarrow \text{'capacity\_deficit'}$

Else if  $\phi_p \leq -\theta_{\text{headroom}} \rightarrow \text{'latent\_headroom'}$

Else  $\rightarrow \text{'neutral'}$

All arithmetic uses the allowed primitives and fixed rounding where required ( $\text{round4}$  for  $\phi_p$ ). The per-pattern  $\Delta E$  aggregation is a simple scan over a bounded number of set rows; we pre-compute daily arrays for each pattern.

 **$u_{\{t+1\}}$  – Control Action**

We derive per-pattern corrections  $u_p = \{d\text{Load}_p, d\text{Set}_p, d\text{Rpe}_p, \text{prefBias}_p\}$  from  $\phi_p$  and flaw class, then enforce safety limits.

First, determine base target RPE for the pattern from the current objective and macroPhase:

$$\text{base\_rpe\_wave} = \text{SCHEMES}[\text{objective}].\text{rpeWave} \text{ // } [w_1, w_2, w_3]$$

$$\text{phase\_rpe\_mod} = \text{PHASE\_MODS}[\text{macroPhase}].\text{rpe}$$

The block's prescribed RPE for each week  $w = \text{base\_rpe\_wave}[w] + \text{phase\_rpe\_mod}$  (clamped later).

We'll treat the correction  $d\text{Rpe}_p$  as a delta added to those values before clamping.

Controller mapping (using  $\phi_p$  thresholds):

If flawClass is 'caution' or an active restrictive guardrail exists ( $\text{load\_multiplier} < 1$ ,  $\text{set\_delta} < 0$ ,  $\text{rpe\_cap\_max} < 10$ ) or halt flag true:

```
// only reductions allowed
dRpe_p = min(0, raw_dRpe)
dSet_p = min(0, raw_dSet)
dLoad_target = min(1.0, raw_dLoad_target)
prefBias_p = min(0, raw_prefBias)
Else, raw deltas from ϕ_p via deadband and thresholds:
```

- If  $|\phi_p| < 0.15 \rightarrow$  no action: raw\_dRpe=0, raw\_dSet=0, raw\_dLoad\_target=1.0, raw\_prefBias=0.
- Else if  $\phi_p \geq 0.4$ : raw\_dRpe = -0.5, raw\_dSet = -1, raw\_dLoad\_target = 0.95, raw\_prefBias = -1.
- Else if  $\phi_p \geq 0.15$ : raw\_dRpe = -0.5, raw\_dSet = -1, raw\_dLoad\_target = 1.0, raw\_prefBias = -1.
- Else if  $\phi_p \leq -0.4$ : raw\_dRpe = +0.5, raw\_dSet = +1, raw\_dLoad\_target = 1.05, raw\_prefBias = +1.
- Else if  $\phi_p \leq -0.15$ : raw\_dRpe = +0.5, raw\_dSet = +1, raw\_dLoad\_target = 1.0, raw\_prefBias = +1.

After safety cap, snap dLoad\_target to nearest literal in LOAD\_MODIFIER\_LITERALS (choose lower if tie)  $\rightarrow$  dLoad\_p.

dSet\_p = clamped raw\_dSet (already integer, but ensure max -1).

dRpe\_p = clamp(raw\_dRpe) to  $\pm 0.5$ , and further clamped so that final target\_rpe  $\in [5.0, \text{base\_rpe\_cap}]$ , where base\_rpe\_cap from profile.

prefBias\_p = raw\_prefBias.

Global volume budget: let PosSetPatterns =  $\{ p \mid dSet_p > 0 \}$ . If  $\sum dSet_p > \text{MAX\_ADDED\_SETS}$ , sort PosSetPatterns by  $\phi_p$  ascending (most negative first) and set dSet\_p = 0 for patterns beyond the first MAX\_ADDED\_SETS (keep only up to 2 additions).

Daily AdjustmentVector projection:

Given the set of movement patterns scheduled on a specific day (DayPatterns  $\subseteq$  MovementPattern), derive a session-level AdjustmentVector:

load\_mod =  $\min_{p \in \text{DayPatterns}} dLoad_p$  (if none, 1.0)

set\_mod =  $\min_{p \in \text{DayPatterns}} dSet_p$

rpe\_cap =  $\min_{p \in \text{DayPatterns}} \text{clamp}(\text{base\_rpe\_cap} + dRpe_p, 5.0, 10.0)$

then round to nearest 0.5 ( $\text{round}(\text{value} * 2) / 2$ ) and snap to nearest literal in RPE\_CAP\_LITERALS.

coaching\_cue = deterministic text from the most severe active flaw (e.g., "Reduce effort on hinge due to persistent overexertion."). Use pattern with max  $|\phi_p|$ .

ClampAdjustment is applied as defined, but our derived values already lie in the literal sets.

Now, compile constants and invariants.

</thinking>

## F — Flaw-Detection Operator



## Closed-Form Formulas

Let  $W = 21$  trailing calendar days, indexed  $i = 0, 1, \dots, 20$  where  $i = 0$  is the oldest day and  $i = 20$  is the current day  $t$ .

For each movement pattern  $p \in \text{MOVEMENT\_PATTERNS}$ , define the following daily arrays from the window:

- $n_p(i)$  = number of sets logged for pattern  $p$  on day  $i$  (0 if no session).
- If  $n_p(i) > 0$ :

$$e_p(i) = \frac{1}{n_p(i)} \sum_{\text{sets on day } i} (\text{rpe\_actual} - \text{target\_rpe})$$

$$w_p(i) = \frac{1}{n_p(i)} \sum_{\text{sets on day } i} \frac{1}{\max(1, \text{effective\_load\_kg} / \max(\text{base\_load\_kg}, 0.01))}$$

If  $\text{base\_load\_kg} \leq 0$  the ratio is taken as 1.

- $j_p(i)$  = maximum severity over niggle rows on day  $i$  with  $\text{region} \in \text{PATTERN\_JOINTS}[p]$ , or 0 if none.

## 1. Exponential Moving Sum (EMA)

Choose decay factor  $\lambda \in (0, 1)$ .

Weight for day  $i$ :  $\omega_i = \lambda^{20-i}$ .

Normalization constant:

$$S_{\max} = \sum_{k=0}^{20} \lambda^k = \frac{1 - \lambda^{21}}{1 - \lambda}$$

## 2. Deficit / Headroom Accumulation

$$P_p = \sum_{\substack{i=0 \\ e_p(i) > 0}}^{20} \omega_i \cdot w_p(i) \cdot \min(e_p(i), E_{\max}) \cdot \max(0, 1 - \frac{j_p(i)}{J_{\max}})$$

$$N_p = \sum_{\substack{i=0 \\ e_p(i) < 0}}^{20} \omega_i \cdot w_p(i) \cdot \min(|e_p(i)|, E_{\max})$$

$E_{\max}$  caps the RPE error magnitude;  $J_{\max} = 10$  (max niggle severity).

## 3. Base Deficit Score

$$D_p = P_p - N_p, \quad D_{\text{norm}} = \frac{D_p}{E_{\max} \cdot S_{\max}}$$

Apply deadband:

$$\phi_{\text{base}} = \begin{cases} 0, & |D_{\text{norm}}| < \delta \\ D_{\text{norm}}, & \text{otherwise} \end{cases}$$

## 4. Trend Signal

$$e_{\text{old}} = \frac{1}{|\{i \in [0, 6] \mid n_p(i) > 0\}|} \sum_{\substack{i=0 \\ n_p(i) > 0}}^6 e_p(i), \quad e_{\text{recent}} = \frac{1}{|\{i \in [14, 20] \mid n_p(i) > 0\}|} \sum_{\substack{i=14 \\ n_p(i) > 0}}^{20} e_p(i)$$

(empty sets  $\rightarrow 0$ ).

$$t_p = \tanh\left(\frac{e_{\text{recent}} - e_{\text{old}}}{T_{\text{scale}}}\right) \quad \text{with } \tanh(x) = \frac{e^{2x} - 1}{e^{2x} + 1}$$

## 5. Final Flaw Score

$$\phi_p = \text{clamp}(w_{\text{base}} \cdot \phi_{\text{base}} + w_{\text{trend}} \cdot t_p, -1, 1)$$

Round to 4 decimal places using `round4`.

## 6. Observations & Classification

$$\text{obs}_p = |\{i \mid n_p(i) > 0\}|$$

If  $\text{obs}_p < \text{MIN\_OBSERVATIONS} \rightarrow \text{flawClass} = \text{'neutral'}$ .

Else if  $\max_i j_p(i) \geq \text{trriageMin}(\text{trainingAge}) \rightarrow \text{'caution'}$ .

Else if  $\phi_p \geq \theta_{\text{def}} \rightarrow \text{'capacity_deficit'}$ .

Else if  $\phi_p \leq -\theta_{\text{headroom}} \rightarrow \text{'latent_headroom'}$ .

Else  $\rightarrow \text{'neutral'}$ .

## TypeScript Interface & Function Signatures

```
ts

/**
 * Flaw classification per movement pattern.
 */
export type FlawClass = 'capacity_deficit' | 'latent_headroom' | 'caution' | 'neutral';

/**
 * Per-pattern flaw signal produced by the detector.
 */
export interface PatternFlaw {
 pattern: MovementPattern;
 phi: number; // round4, ∈ [-1, 1]
 flawClass: FlawClass;
 observations: number; // count of days with ≥1 set in window
}

/**
 * Top-level flaw report.
 */
export interface FlawReport {
 patterns: Record<MovementPattern, PatternFlaw>;
 windowSummary: {
 dominantFlaw: FlawClass;
 maxAbsPhi: number;
 };
}
```

```
// The pattern with the largest |phi|
criticalPattern: MovementPattern | null;
};
// Any active global guardrail from the most recent restrictive report.
globalGuardrail: Guardrail | null;
}

/**
 * Deterministic Flaw-Detection Operator F.
 * Consumes at most 21 daily StateVectorRows, per-pattern ΔE summaries,
 * niggle rows, and the athlete's training age for triage thresholds.
 *
 * @param stateVectors - 21 consecutive daily rows (oldest first).
 * @param patternDailyDelta - per pattern, per day, {avgDeltaRPE: number|null, avgAttenua
tion: number, setCount: number, maxJointSev: number}.
 * @param trainingAge - beginner | intermediate | advanced | elite.
 * @returns FlawReport with deterministic φ and classes.
 */
export function detectFlaws(
 stateVectors: readonly StateVectorRow[], // length ≤ 21
 patternDailyDelta: Record<MovementPattern, {
 avgDeltaRPE: (number | null)[]; // length = stateVectors.length, null if no sets
 avgAttenuation: number[]; // 0 if no sets
 setCount: number[]; // integer
 maxJointSev: number[]; // 0..10
 }>,
 trainingAge: TrainingAge
): FlawReport;
```

u\_{t+1} — Control Action Operator

Closed-Form Mapping

Inputs: FlawReport , UserProfile (containing trainingAge , objective , base\_rpe\_cap , macroPhase ), and optionally the set of patterns for a specific day to produce a daily AdjustmentVector .

1. Per-Pattern Raw Corrections

Let φ = φ\_p. Determine raw deltas using deadband and thresholds:

Condition	raw_dRpe	raw_dSet	raw_dLoad_target	raw_prefBias	
flawClass == 'caution'					
OR any restrictive guardrail OR halt (see safety override below)	—	—	—	—	
(	$\phi$	< 0.15)	0	0	1.00

Condition	raw_dRpe	raw_dSet	raw_dLoad_target	raw_prefBias
$\phi \geq 0.4$	-0.5	-1	0.95	-1
$0.15 \leq \phi < 0.4$	-0.5	-1	1.00	-1
$\phi \leq -0.4$	+0.5	+1	1.05	+1
$-0.4 < \phi \leq -0.15$	+0.5	+1	1.00	+1

If `flawClass == 'caution'` or `globalGuardrail` is restrictive ( `load_multiplier < 1` or `set_delta < 0` or `rpe_cap_max < 10` ) or any loaded joint for pattern has severity  $\geq$  `trriageMin` or `halt true`, then force:

- `dRpe_p = min(0, raw_dRpe)`
- `dSet_p = min(0, raw_dSet)`
- `dLoad_target = min(1.0, raw_dLoad_target)`
- `prefBias_p = min(0, raw_prefBias)`

Otherwise `dRpe_p = raw_dRpe` , etc.

## 2. Snap to Literal Domains

- `dLoad_p = snapToLiteral(dLoad_target, LOAD_MODIFIER_LITERALS)` (nearest, tie  $\rightarrow$  lower).
- `dSet_p = clamp(raw_dSet, -1, 1)` (already integer).
- `dRpe_p = clamp(raw_dRpe, -0.5, 0.5)` and further ensure that for each week  $w \in \{1, 2, 3\}$ ,

$$\text{target\_rpe}_{p,w} = \text{clamp}(\text{SCHEMES}[\text{objective}].\text{rpeWave}[w] + \text{PHASE\_MODS}[\text{n}])$$



Round to 0.5.

- `prefBias_p = raw_prefBias` (already  $\in \{-1, 0, 1\}$ ).

## 3. Global Volume Budget

Let  $\mathcal{P}^+ = \{p \mid dSet_p > 0\}$ .

If  $\sum_{p \in \mathcal{P}^+} dSet_p > \text{MAX\_ADDED\_SETS}$ , sort  $\mathcal{P}^+$  by  $\phi_p$  ascending (most headroom first) and set `dSet_p = 0` for all but the first `MAX_ADDED_SETS` patterns.

## 4. Daily Adjustment Vector Projection

For a given day with scheduled patterns `DayPatterns`  $\subseteq$  `MOVEMENT_PATTERNS` :

$$\begin{aligned} \text{load\_mod} &= \min_{p \in \text{DayPatterns}} dLoad_p \quad (\text{default } 1.0) \\ \text{set\_mod} &= \min_{p \in \text{DayPatterns}} dSet_p \\ \text{rpe\_cap} &= \min_{p \in \text{DayPatterns}} \text{clamp}(\text{base\_rpe\_cap} + dRpe_p, 5.0, 10.0) \end{aligned}$$

Then `rpe_cap = snapToLiteral(roundToHalf(rpe_cap), RPE_CAP_LITERALS)` .  
`coaching_cue` is generated deterministically from the pattern with maximal  $|\phi_p|$  and its  
 flaw class (e.g., "Reduce effort on hinge – persistent capacity deficit").  
 Finally apply `clampAdjustment` (idempotent).

## TypeScript Interface & Function Signatures

```
ts

/** Per-pattern corrective perturbations for the next 4-week block. */
export interface PatternCorrection {
 dLoad_p: number; // snapped to LOAD_MODIFIER_LITERALS
 dSet_p: number; // integer ∈ [-1, 1]
 dRpe_p: number; // ∈ [-0.5, 0.5]
 prefBias_p: -1 | 0 | 1; // biases pattern substitution router
}

/** The full control action for the upcoming block. */
export interface ControlAction {
 corrections: Record<MovementPattern, PatternCorrection>;
 blockAddedSets: number; // total positive set additions (≤ MAX_ADDED_SETS)
}

/**
 * Derives the block-level auto-correction from a FlawReport.
 *
 * @param report - deterministic flaw detection output.
 * @param profile - user's training age, objective, base_rpe_cap, current macroPhase.
 * @returns a deterministic ControlAction.
 */
export function deriveControlAction(
 report: FlawReport,
 profile: UserProfile & { objective: Objective; macroPhase: MacroPhase }
): ControlAction;

/**
 * Converts the block corrections into a daily AdjustmentVector for a given day's pattern
 list.
 * Guaranteed to lie inside the literal domains after clampAdjustment.
 */
export function deriveDailyAdjustment(
 action: ControlAction,
 dayPatterns: readonly MovementPattern[],
 profile: UserProfile
): AdjustmentVector;
```

## Constant Tables

All constants are human-reviewable and mirror the style of `PHASE_MODS` /  
`EXPERIENCE_TRIAGE` .

```
ts
```

```

/**
 * FLAW_DETECTION_CONSTANTS
 * -----
 * Fixed parameters for the flaw-detection operator F.
 */
export const FLAW_DETECTION_CONSTANTS = {
 /** Decay factor λ for exponential moving sum ($0 < \lambda < 1$). */
 LAMBDA: 0.88,
 /** Cap on absolute ΔE magnitude (RPE error). */
 E_MAX: 3.0,
 /** Maximum niggle severity (joint scale). */
 J_MAX: 10,
 /** Deadband for ϕ_{base} : $|D_{\text{norm}}| < \delta \rightarrow 0$. */
 DELTA: 0.15,
 /** Weight for the base accumulation term. */
 W_BASE: 0.7,
 /** Weight for the trend term. */
 W_TREND: 0.3,
 /** Scale factor inside tanh for trend. */
 T_SCALE: 1.0,
 /** Minimum days with at least one set to trust the signal. */
 MIN_OBSERVATIONS: 5,
 /** ϕ threshold for 'capacity_deficit' class. */
 THETA_DEFICIT: 0.3,
 /** ϕ threshold for 'latent_headroom' class (absolute value). */
 THETA_HEADROOM: 0.3,
} as const;

/**
 * CONTROL_AUTHORITY_CONSTANTS
 * -----
 * Bounds on how much the autopilot may change the next block.
 */
export const CONTROL_AUTHORITY = {
 /** Maximum total positive set additions across all patterns in one block. */
 MAX_ADDED_SETS: 2,
 /** Allowed load modifier step (one literal step up/down). */
 LOAD_STEP: 0.05, // derived from LOAD_MODIFIER_LITERALS adjacent differences
 /** Allowed RPE delta per block. */
 RPE_STEP: 0.5,
 /** Allowed set delta per pattern. */
 SET_STEP: 1,
 /** Deadband for ϕ : $|\phi| < \text{DEADBAND} \rightarrow \text{no action}$. */
 DEADBAND: 0.15,
 /** ϕ threshold for strong deficit / headroom. */
 STRONG_THRESHOLD: 0.4,
} as const;

/**
 * DEFAULT_LOAD_TARGET
 * Neutral load modifier literal.
 */
export const NEUTRAL_LOAD_MODIFIER = 1.0;

/**
 * LOAD_MODIFIER_LITERALS (shipped, reused)
 */
export const LOAD_MODIFIER_LITERALS = ['0.80', '0.85', '0.90', '0.95', '1.00', '1.05'] as const;

```

```

/**
 * RPE_CAP_LITERALS (shipped, reused)
 */
export const RPE_CAP_LITERALS = ['6.5', '7.0', '7.5', '8.0', '8.5', '9.0', '9.5', '10.0'] as const;

/**
 * Snap a numeric value to the nearest literal in the sorted array (ties → lower).
 * Pure, deterministic, no library calls.
 */
export function snapToLiteral(value: number, literals: readonly string[]): number;

```

## Invariant List (Machine-Checkable Guarantees)

### 1. Determinism

Every arithmetic operation uses only the permitted primitives and fixed rounding rules ( `round2` , `round4` , `roundToHalf` ). No clock, random, or order-dependent reduction. Same persisted inputs → byte-identical `FlawReport` and `ControlAction` .

### 2. Monotone-Conservative under Flags

If any `Guardrail` is restrictive ( `load_multiplier < 1` , `set_delta < 0` , `rpe_cap_max < 10` ) or any loaded joint has niggles severity  $\geq$  `triageMin` , the control action never increases load, sets, or RPE for the affected patterns. It may only reduce them or leave them neutral.

### 3. Domain Closure

- `dLoad_p` is snapped to one of `LOAD_MODIFIER_LITERALS` .
- `dSet_p`  $\in \{-1, 0, 1\}$  (subset of `SET_MODIFIER_LITERALS` ).
- Daily `rpe_cap` is rounded to nearest 0.5 and snapped to `RPE_CAP_LITERALS` .
- The final `AdjustmentVector` passes `clampAdjustment` idempotently.

### 4. Halt Supremacy

If `halt` is true, all corrections become neutral ( `dLoad_p = 1.0` , `dSet_p = 0` , `dRpe_p = 0` , `prefBias_p = 0` ) — no block change is emitted.

### 5. Bounded Authority per Block

- Total positive set additions across all patterns  $\leq$  `MAX_ADDED_SETS` (2).
- Load modifier changes by at most one literal step (0.05) away from 1.0.
- RPE target changes by at most  $\pm 0.5$ .
- Per-pattern set change  $\leq \pm 1$ .  
This ensures the controller cannot windup or oscillate; the existing monotone clamps further limit any daily projection.

### 6. Volume Bifurcation

Corrections act on the prescription (target RPE, sets, load modifier) and use effective tonnage only for condition-weighting inside F. The raw `mech_daily` tonnage / ACWR pipeline is never altered.